
UiA
University of Agder
Norway

Lecture #2.2

Procedural-oriented PLC programming

MAS418

Programming for Intelligent Robotics and Industrial systems

Part II: PLC Software Development

Autumn 2025

Daniel Hagen, PhD

UiA

Previous Lecture

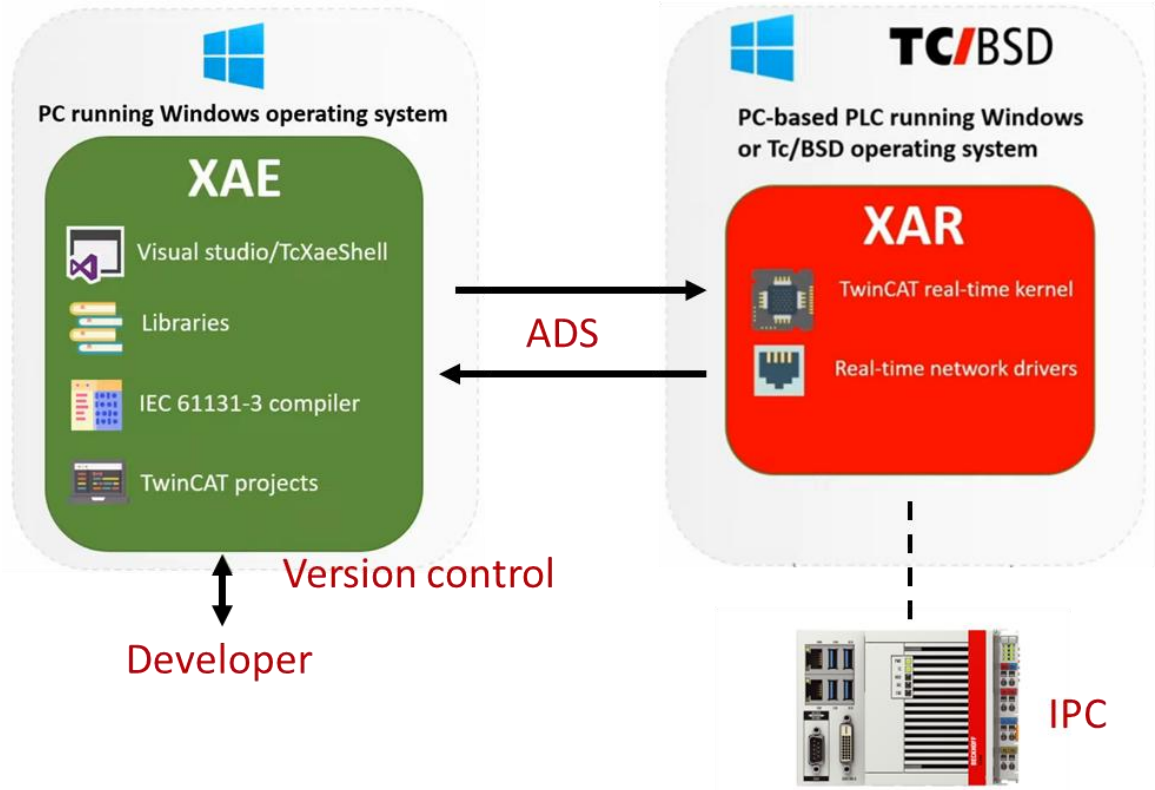
Introduction to Part II -

PLC Software Development:

I. Introduction to PLC programming
- History
- IEC61131-3 standard
- Version-control

II. TwinCAT basics
- Introduction to TwinCAT
- Difference to SW running in OS

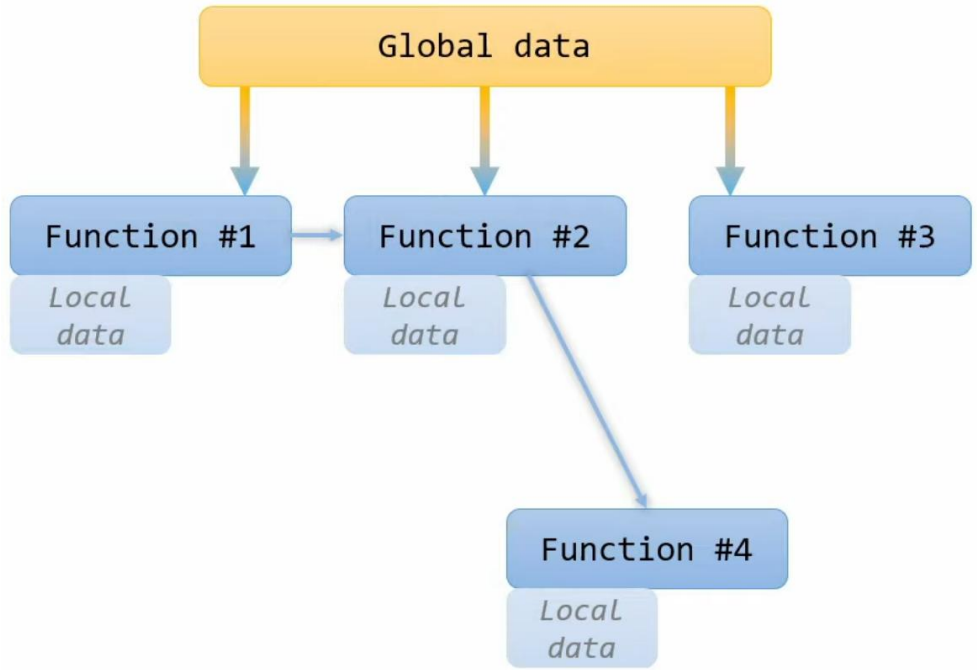
III. Data types & Std. library
- Basic data types
- Time
- Pointers
- Reference
- Arrays
- Enumeration
- Standard library



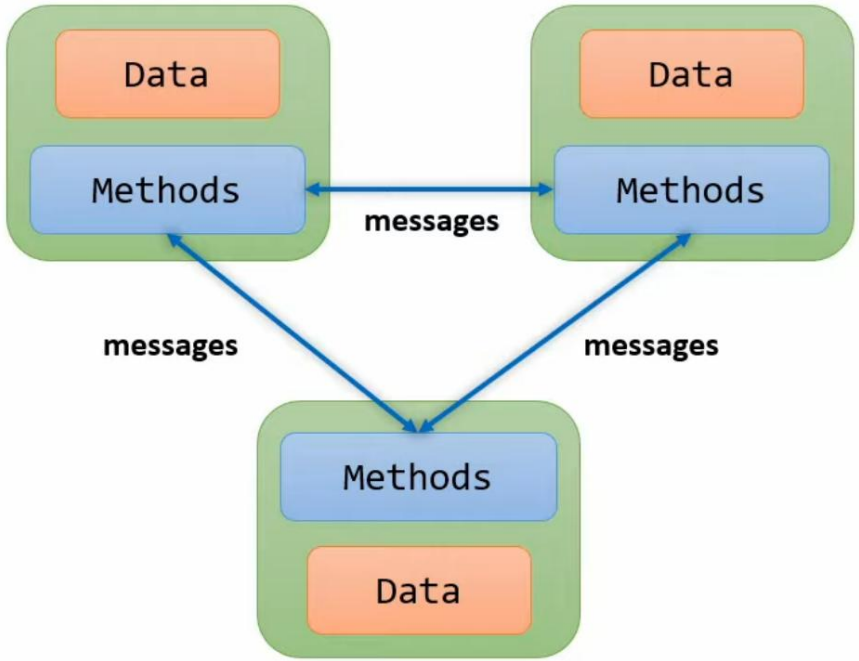
Imperative programming paradigms

- Within the computer science field there are two major imperative programming paradigms:

Procedural Oriented Programming (POP)

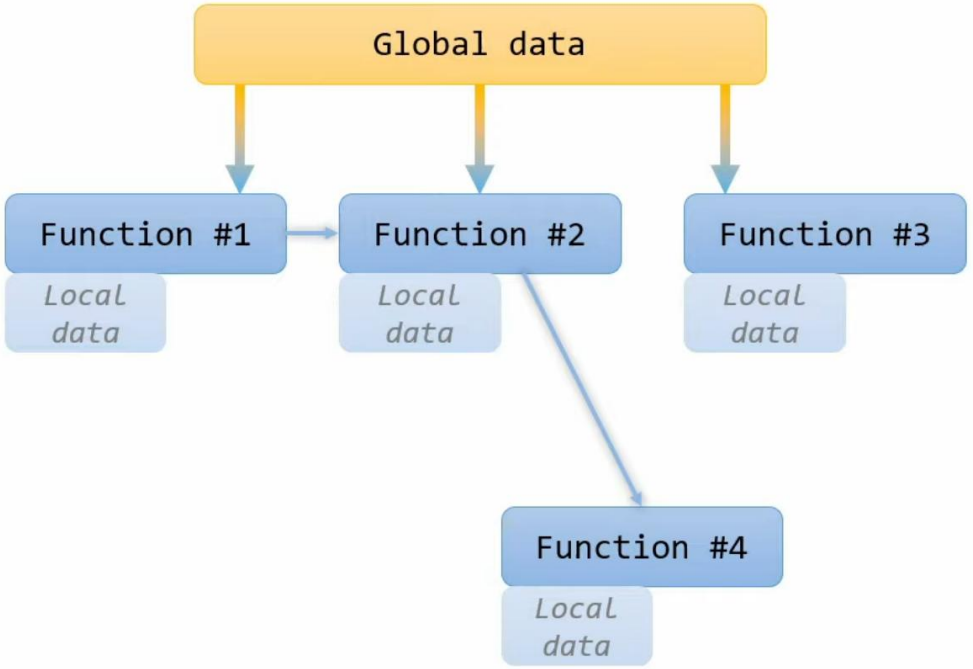


Object Oriented Programming (OOP)



Key takeaways

- **Procedural-oriented PLC programming**
  - Structures
  - Functions
  - Pass by value & pass by reference
  - **Instructions:**
    - Conditional statements
    - CASE instruction
    - FOR loops
    - WHILE loops



Overview

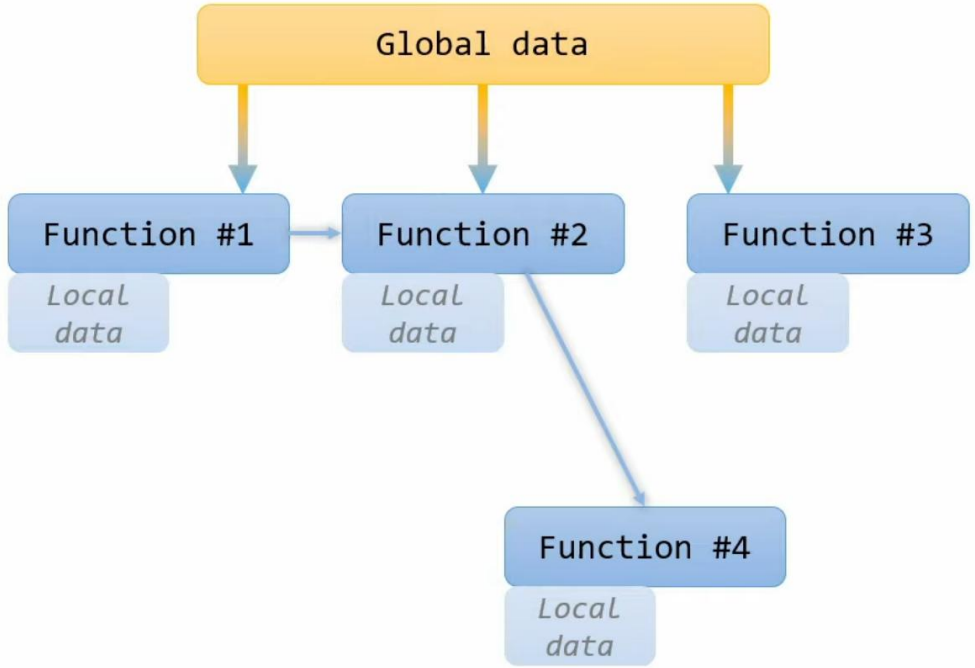
Introduction

Part I: Crane application and simulator

Part II: Structures & Functions

Part III: Instructions

Summary



UiA

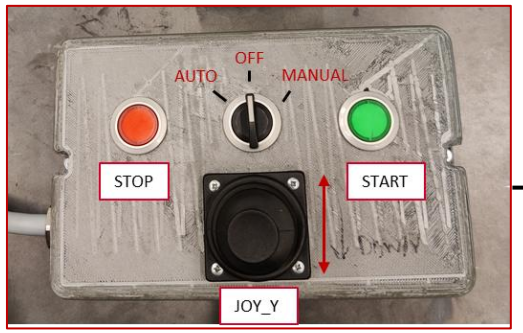
Part I: Crane application and simulator

1. System overview
2. Relevant IO
3. Safety functions
4. Hydraulic cylinder simulator
5. Time-domain simulation
6. Simulink PLC Coder
7. Create new task in TwinCAT

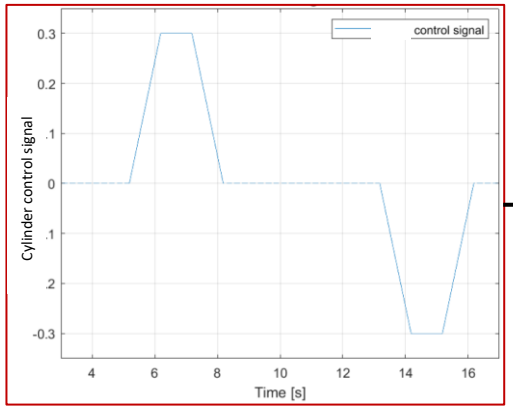
UiA

System overview

Manual Mode



Auto Mode

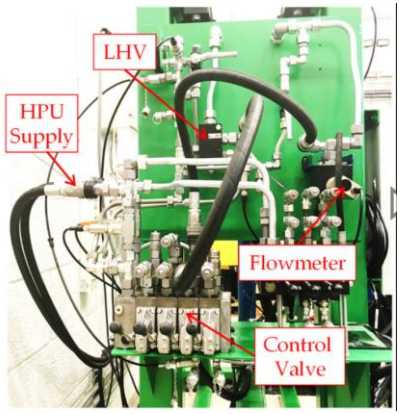


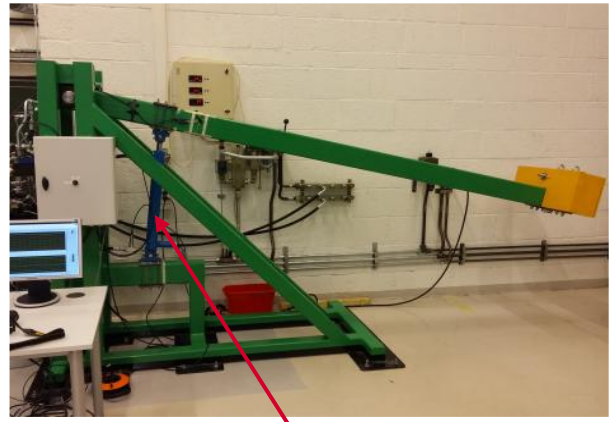
Real-Time Controller (PLC)

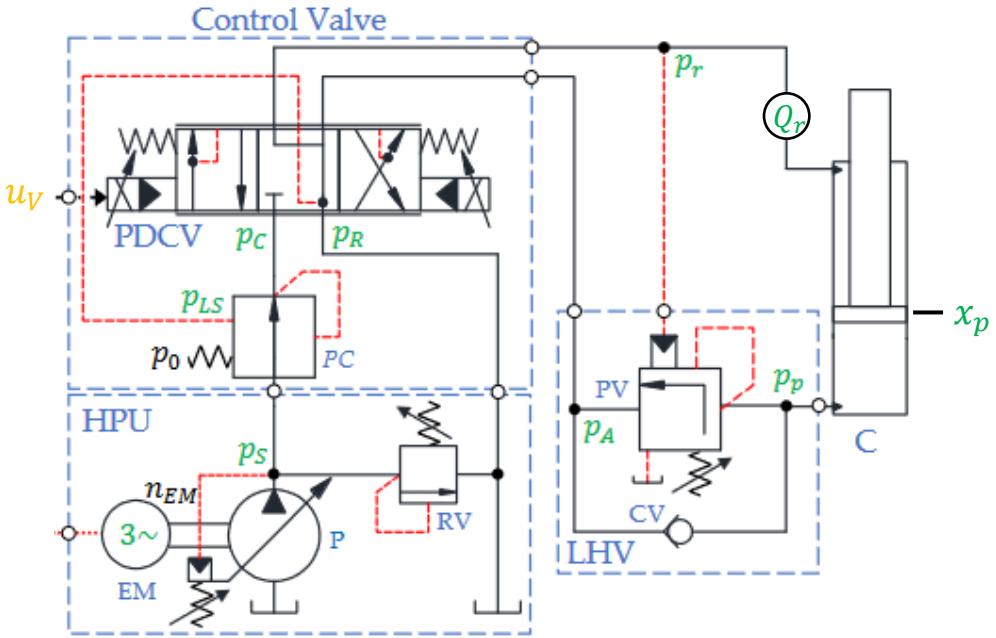
Mechatronics Application

Relevant IO

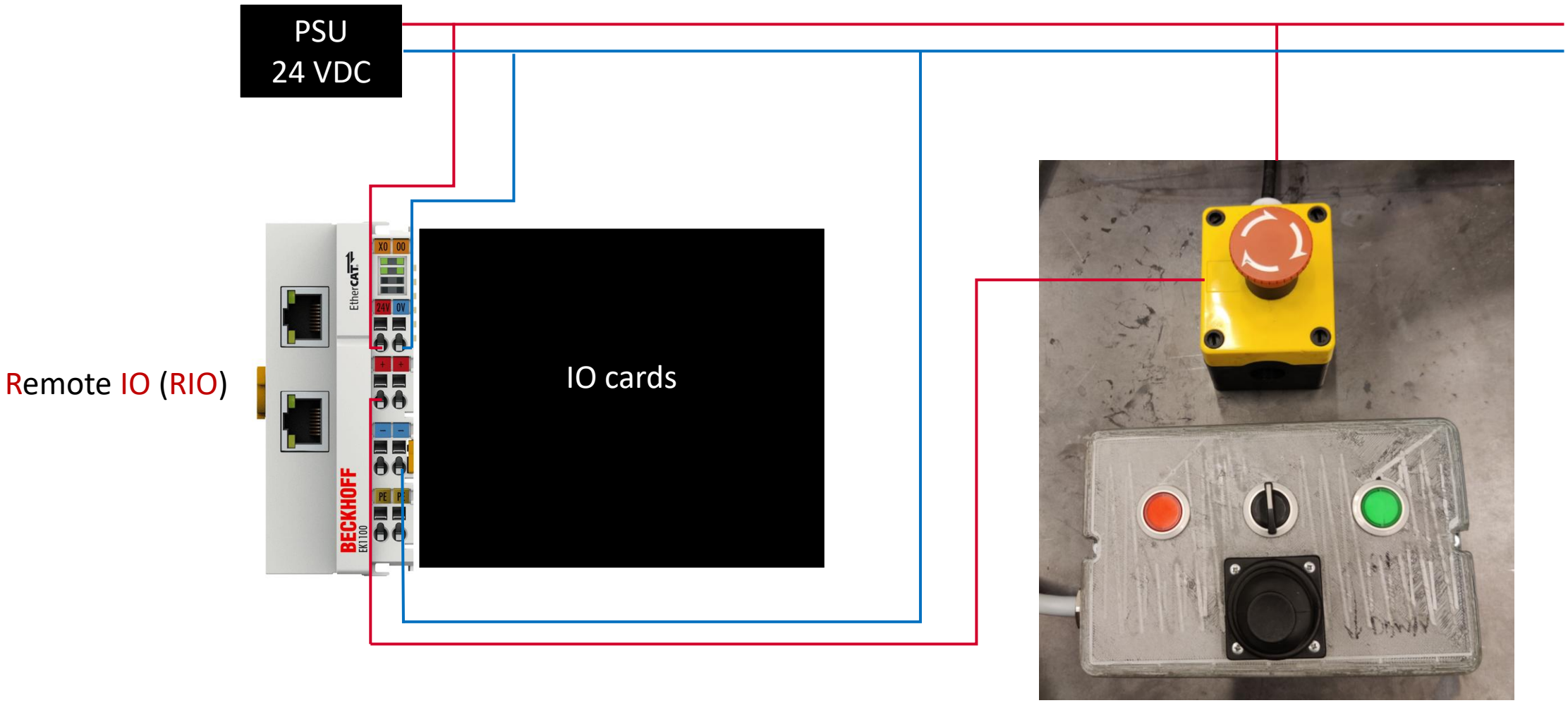
- Cylinder piston position sensor (input: 0…0.5 [m]) - x_p
- Pressure sensors (Input: 0…400 [bar])
  - Supply pressure - p_S
  - Return pressure - p_R
  - Cylinder piston-side pressure - p_p
  - Cylinder rod-side pressure - p_r
  - Pressure between **Control Valve** and **L**oad-**H**olding **V**alve (**LHV**) - p_A
- Flow sensors (Input: -150…150 [l/min])
  - Rod-side cylinder outlet flow - Q_r
- Control Valve (output: -1…1 [-]) - u_V





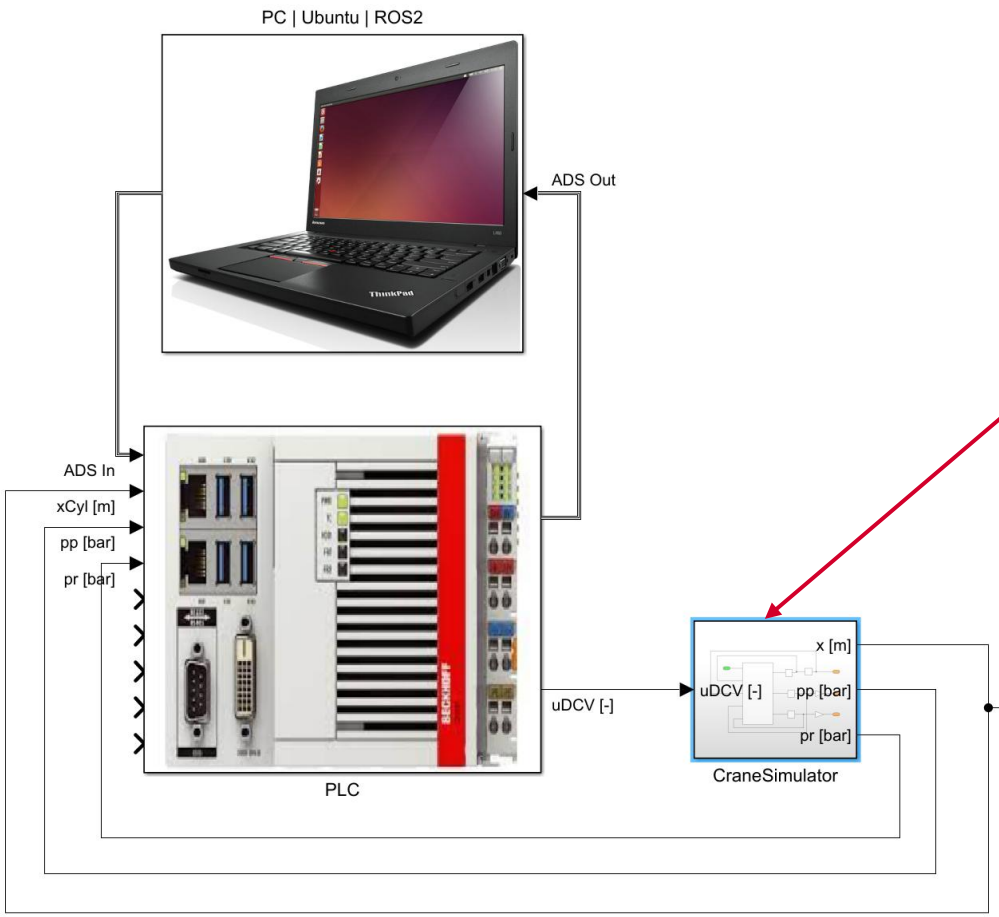


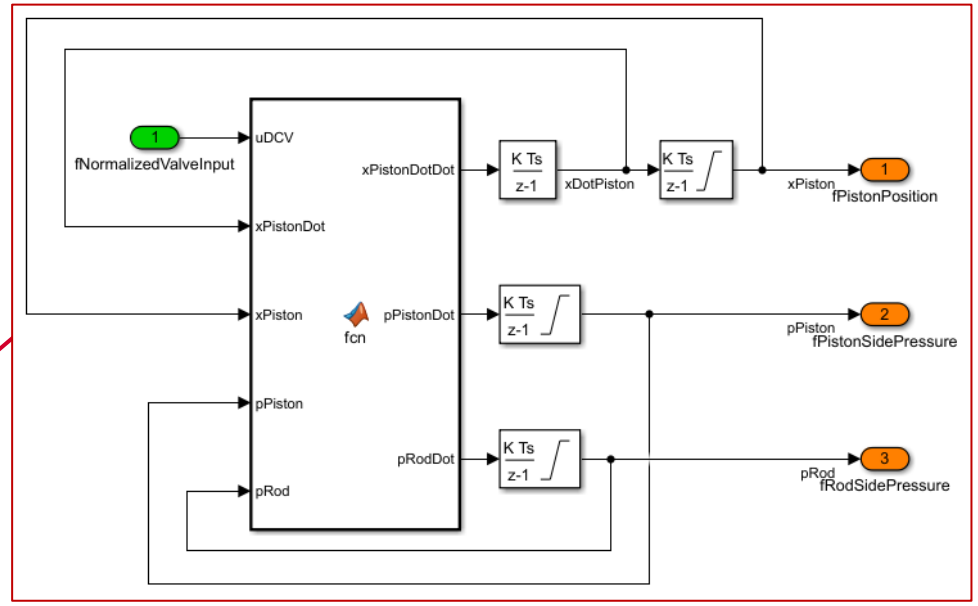
Safety functions



Hydraulic cylinder simulator

- MATLAB model



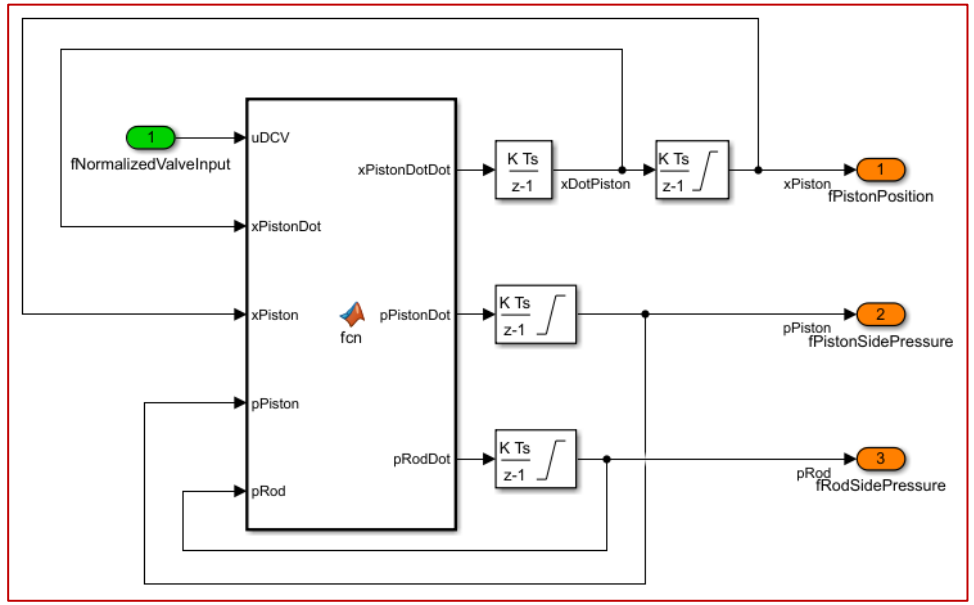


Simplified Hydraulic Cylinder Simulator

See Simulink model for details: https://github.com/DrDanielh/HydraulicCylinderSimulator_SimulinkPlcCoder

Hydraulic cylinder simulator

- MATLAB model



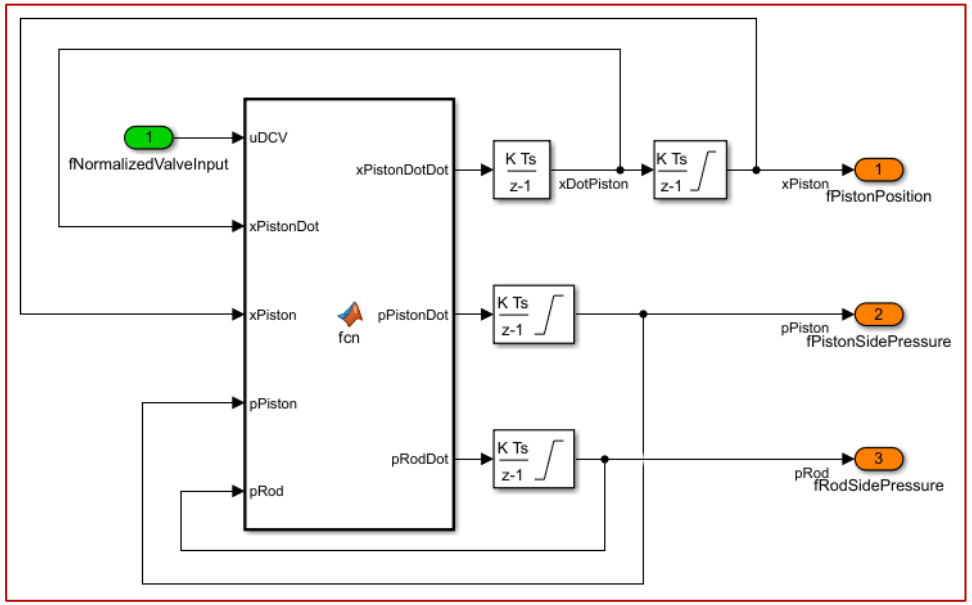

```
function [xPistonDotDot, pPistonDot, pRodDot] = fcn(uDCV, xPistonDot, xPiston, pF
%% Parameters
mEffective = 1.5e+04;  % [kg]
Fexternal = 2.15e+04; % [N]
kFric = 10000; % [N/(ms)]
Apiston = 0.0033; % [m^2]
Aannulus = 0.0023; % [m^2]
beta = 1.0e9; % [Pa]
pSupply = 180e5; % [Pa]
pReturn = 1.1e5; % [Pa]
MaxStroke = 0.5; % [m]
VCyl_A = 1.0e-03;
VCyl_B = 1.25e-03;
p_nom_DCV = 7e5;
Q1_ref_DCV = 19/6e4;
Q2_ref_DCV = 8/6e4;
kv1_DCV = Q1_ref_DCV/sqrt(p_nom_DCV);
kv2_DCV = Q2_ref_DCV/sqrt(p_nom_DCV);
p0 = 7e5; %compensator crack pressure
```

UiA

See Simulink model for details: https://github.com/DrDanielh/HydraulicCylinderSimulator_SimulinkPlcCoder

Hydraulic cylinder simulator

- MATLAB model

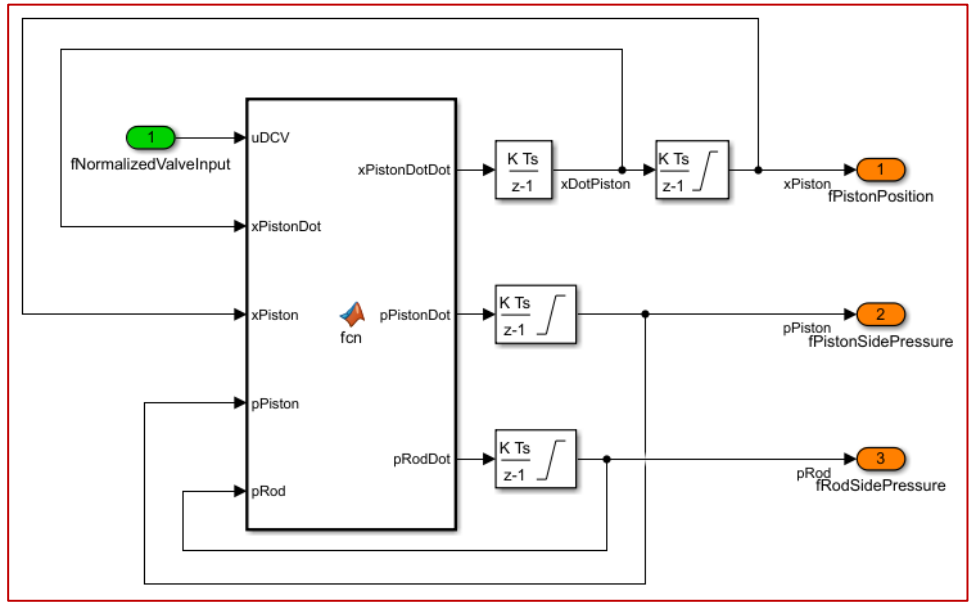



```
20  %% Directional Control Valve - Lifting Cylinder
21  % LS Pressure
22  if uDCV > 0
23       pLS = pPiston;
24   elseif uDCV < 0
25       pLS = pRod;
26   else
27       pLS = 0;
28  end
29
30  if pLS<0
31      pLS = 0;
32  end
33
34  % Pressure Compensator
35  pPC = 0;
36
37  if p0 <= (pSupply - pLS)
38        pPC = pLS+p0;
39    elseif 0 < (pSupply - pLS) < p0
40          pPC = pSupply-pLS;
41    elseif (pSupply - pLS) <= 0
42        pPC = pLS;
43  end
44
45   if pPC<0
46       pPC = 0;
47   end
48  % Orifice Equations
49  if uDCV > 0
50  QDCV_in =  kv1_DCV*uDCV*sign(pPC-pPiston)*sqrt(abs(pPC-pPiston)); % QpistonSide
51  QDCV_out = kv2_DCV*uDCV*sign(pRod-pReturn)*sqrt(abs(pRod-pReturn)); % QrodSide
52  elseif uDCV < 0
53  QDCV_out =  kv1_DCV*uDCV*sign(pPC-pRod)*sqrt(abs(pPC-pRod)); % QrodSide
54  QDCV_in =  kv2_DCV*uDCV*sign(pPiston-pReturn)*sqrt(abs(pPiston-pReturn)); % QpistonSide
55  else
56  QDCV_in = 0;
57  QDCV_out = 0;
58  end
```

See Simulink model for details: https://github.com/DrDanielh/HydraulicCylinderSimulator_SimulinkPlcCoder

Hydraulic cylinder simulator

- MATLAB model



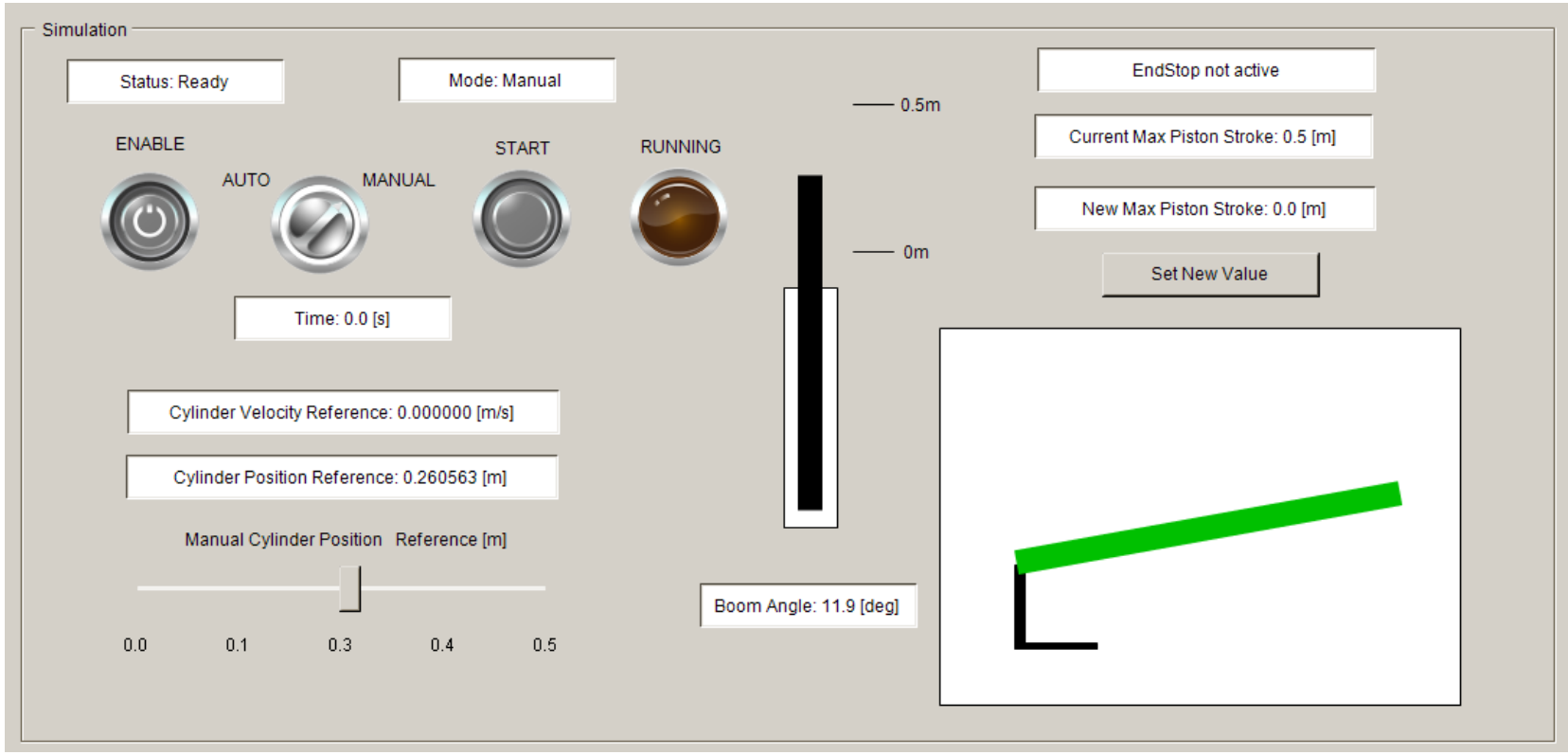

```
59  %% Pressure Gradients
60  VA = VCyl_A + xPiston*Apiston;
61  VB =  VCyl_B + (MaxStroke - xPiston)*Aannulus;
62  VA_dot = Apiston*xPistonDot;
63  VB_dot = Aannulus*xPistonDot;
64  pPistonDot = beta/VA*(QDCV_in - VA_dot);
65  pRodDot = beta/VB*(VB_dot-QDCV_out);
66
67  % Viscouse Friction
68  Ffric = kFric*xPistonDot;
69  %% Hydraulic Force
70  Fhyd = pPiston*Apiston - pRod*Aannulus;
71
72  %% Newton Second Law of Motion
73  xPistonDotDot = (Fhyd - Ffric - Fexternal)/mEffective;
74
```

UiA

See Simulink model for details: https://github.com/DrDanielh/HydraulicCylinderSimulator_SimulinkPlcCoder

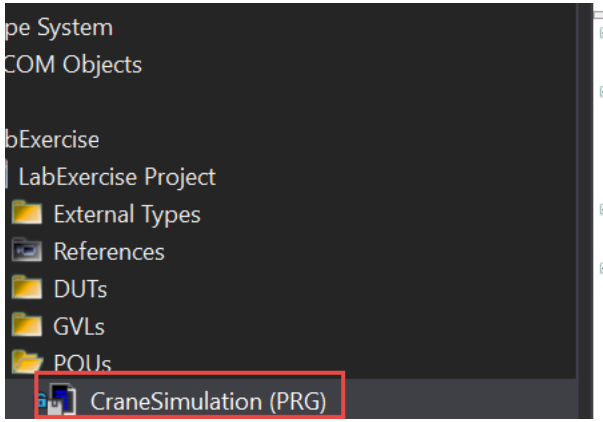
Time-domain simulation

Example of Green Crane



Time-domain simulation

Example of Green Crane

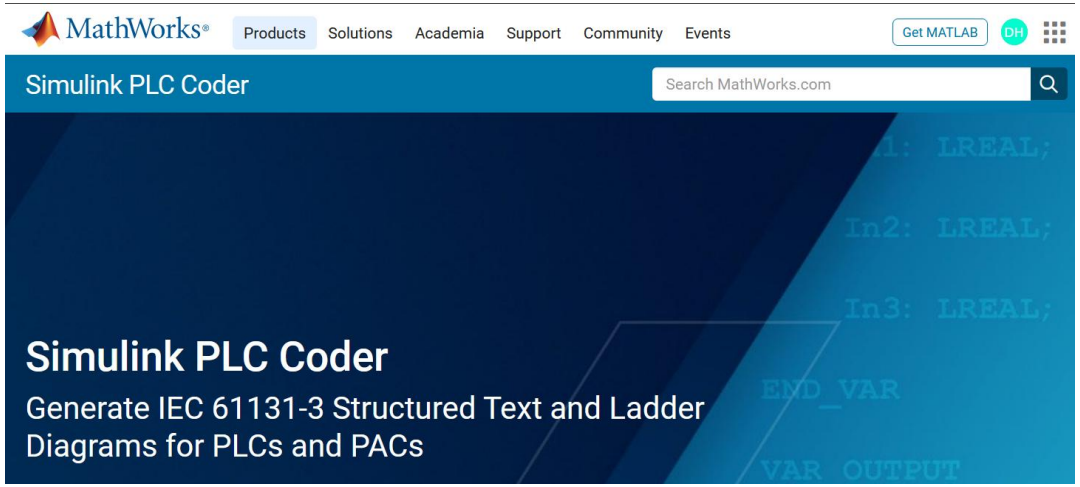



```
IF Inputs.bStartSimulation AND Inputs.bAutoMode  THEN
    fTime := fTime + Parameters.fCyclicTime;
ELSE
    fTime := 0;
END_IF

IF fTime > 0 AND Inputs.bAutoMode THEN
    bRunning := TRUE;
ELSE
    bRunning := FALSE;
END_IF

fPositionAutoMode := Inputs.fCylinderVelocityReference * 1/2*(fTime*fTime) + fInitialVelocity * fTime + fInitialPosition;
```

Simulink PLC Coder

Procedure

- Use a separated Simulink project with specific settings
- Use supported Simulink blocks (i.e. discrete integrators etc.)
- When building new code, delete previous generated folder
- The Simulink blocks need to be in a subsystem



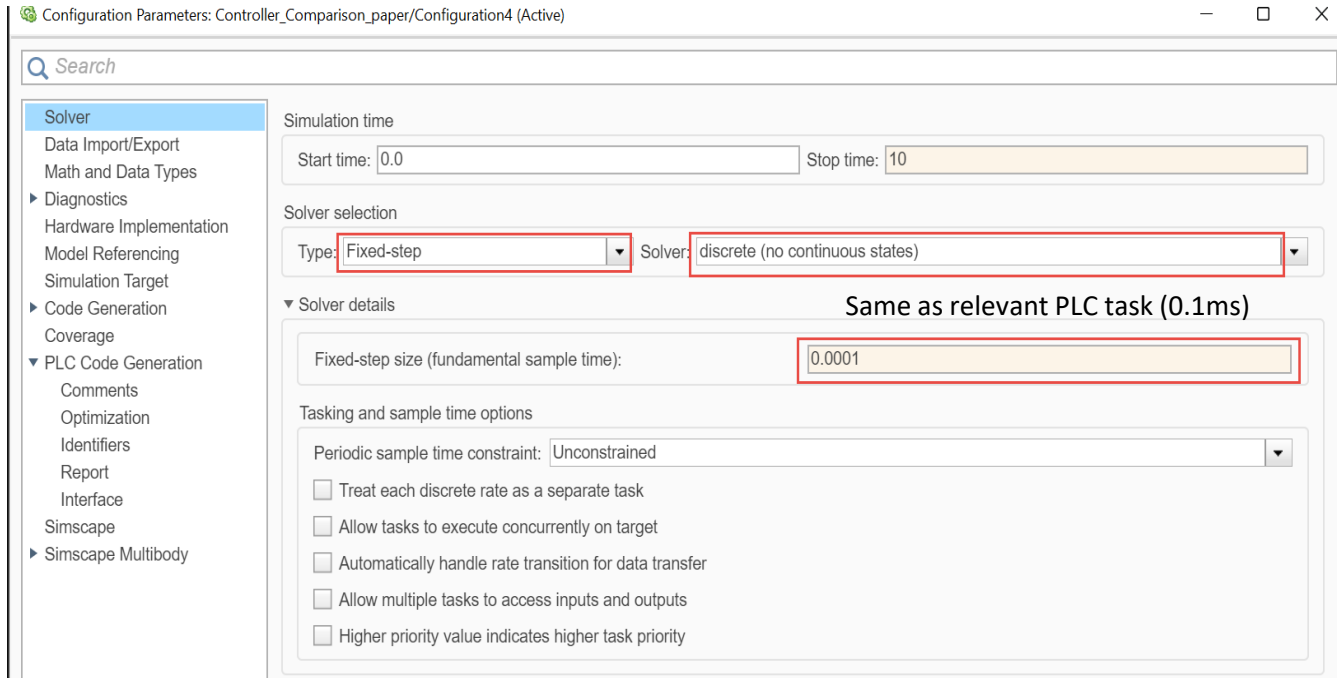
https://se.mathworks.com/products/simulink-plc-coder.html

UiA

Simulink PLC Coder

Simulink settings

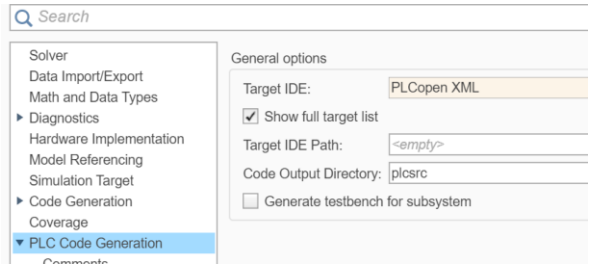
- Solver

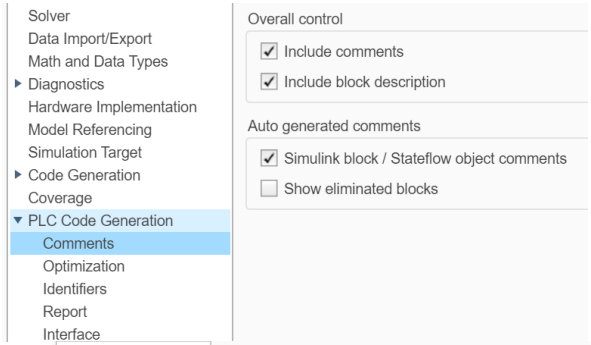


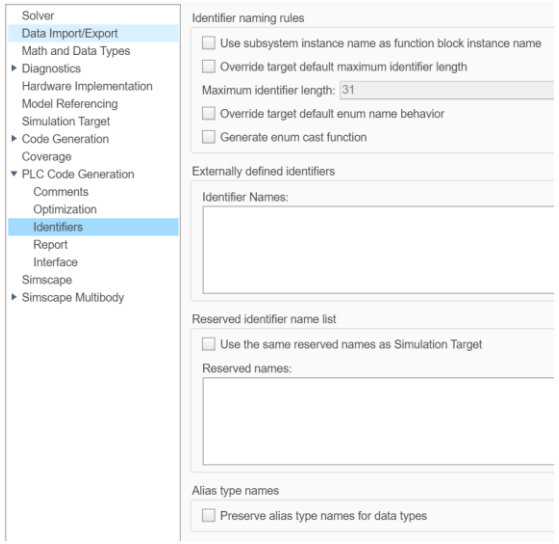
Simulink PLC Coder

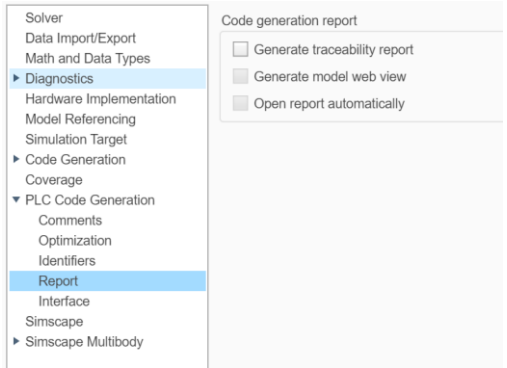
Simulink settings

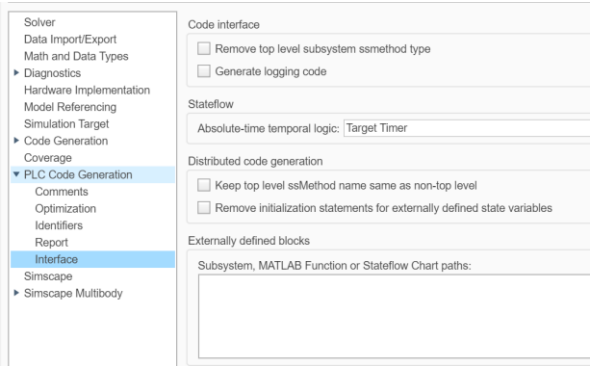
- Solver
- PLC coder options
  - Play with them

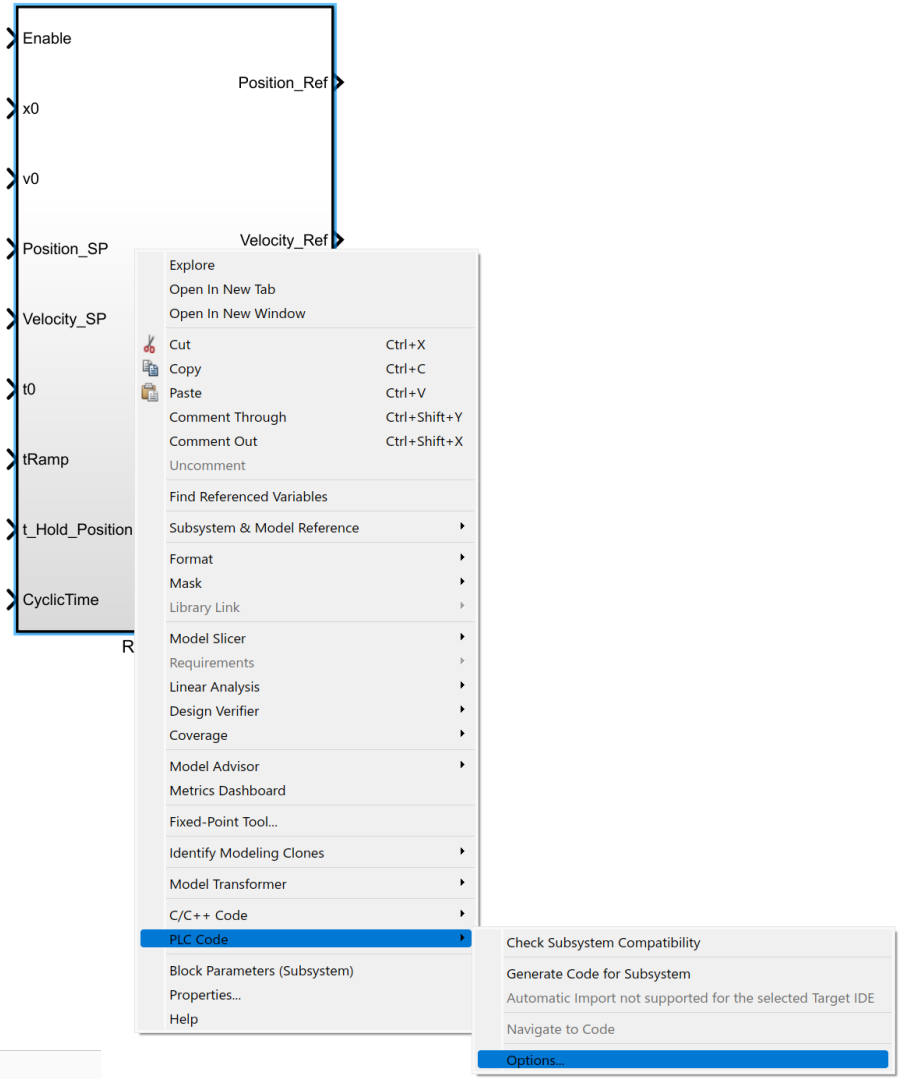








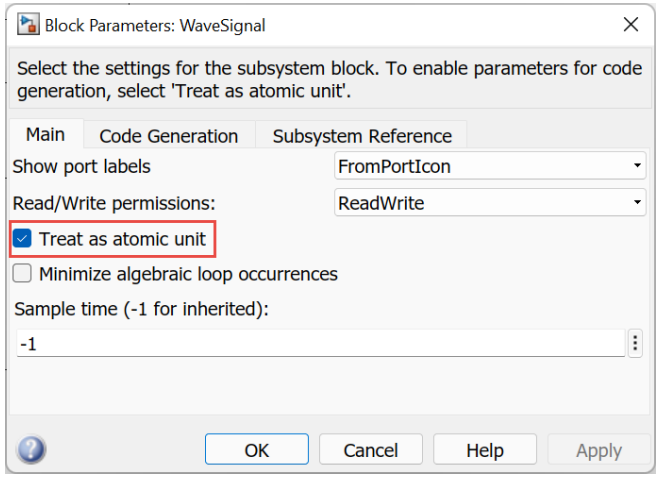


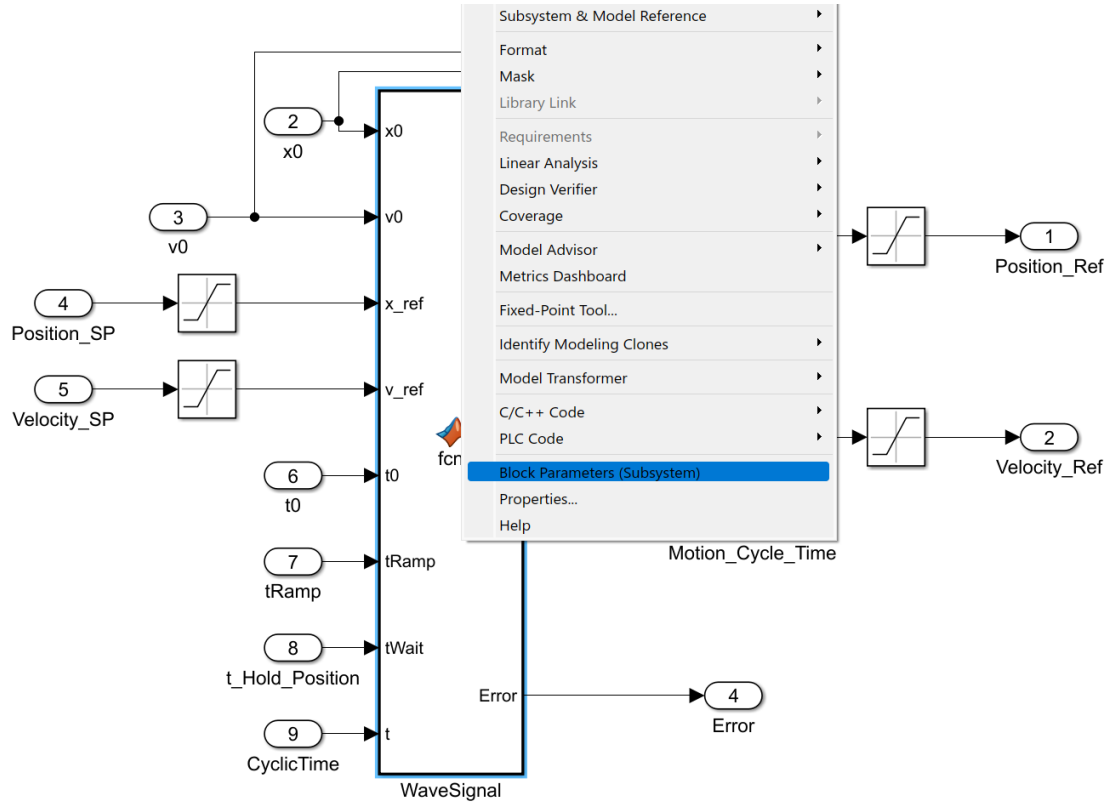


Simulink PLC Coder

Simulink settings

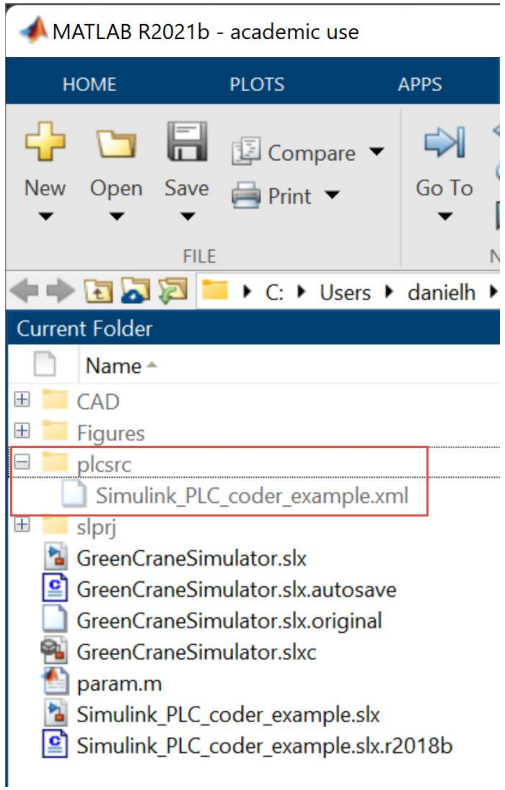
- Solver
- PLC coder options
  - Play with them
- MATLAB function settings

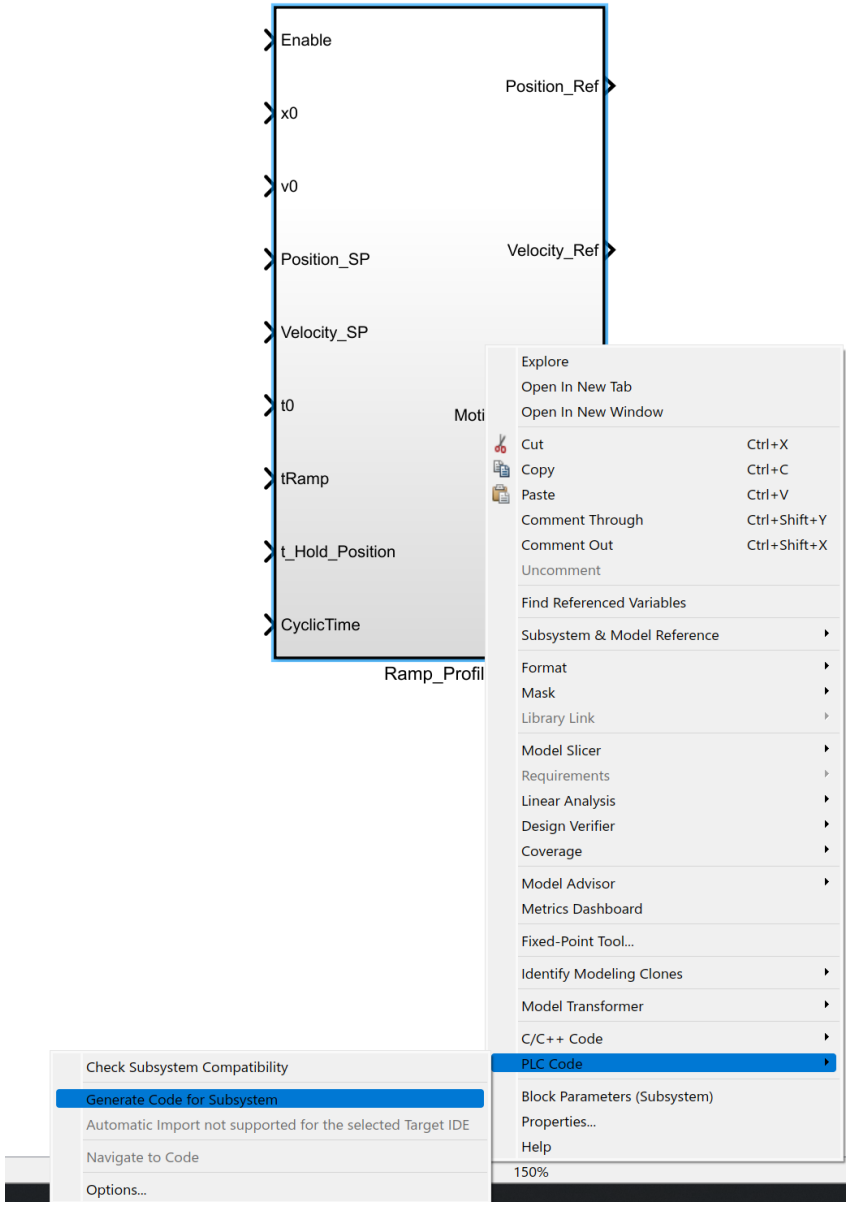




Simulink PLC Coder

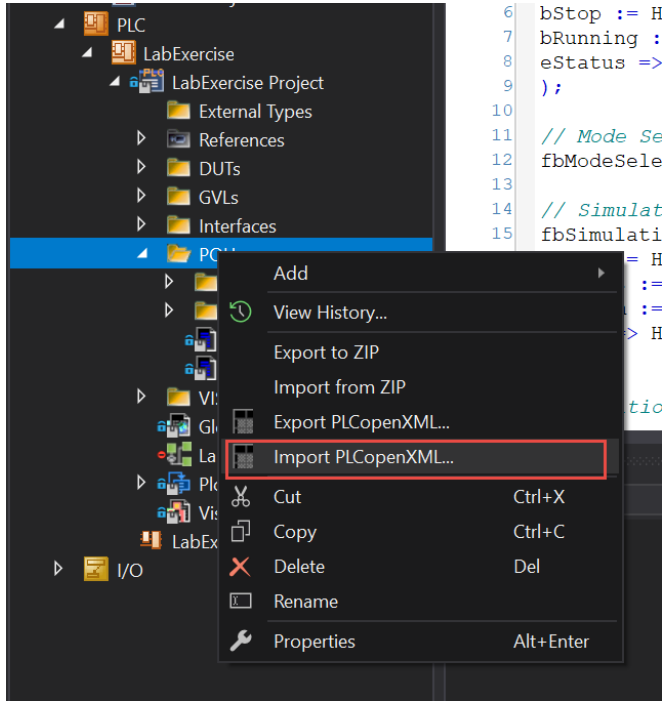
Code generation

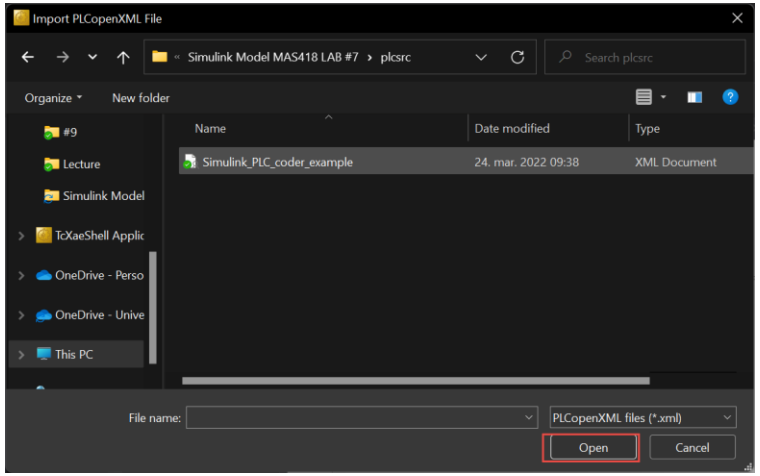


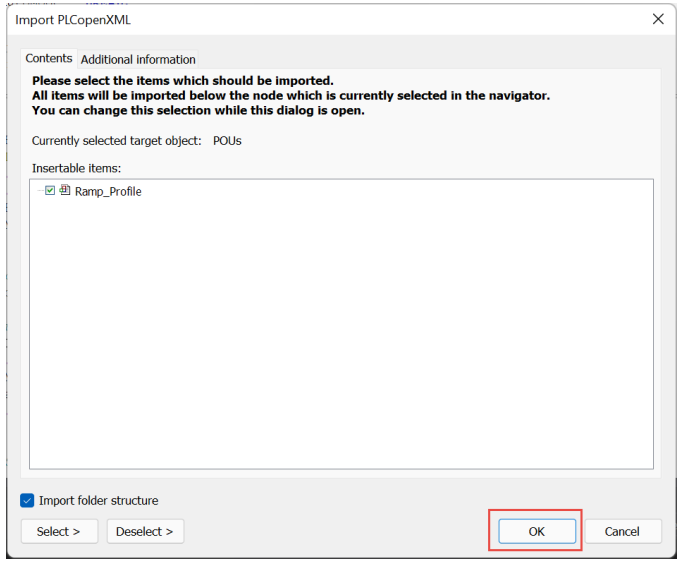


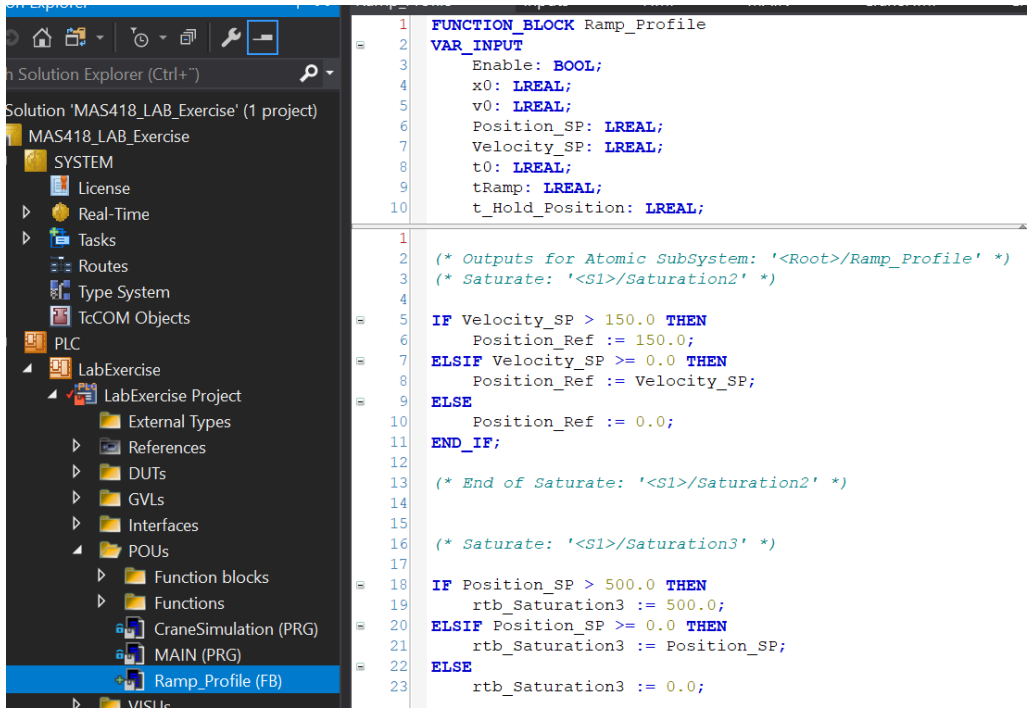
Simulink PLC Coder

Implement code in TwinCAT



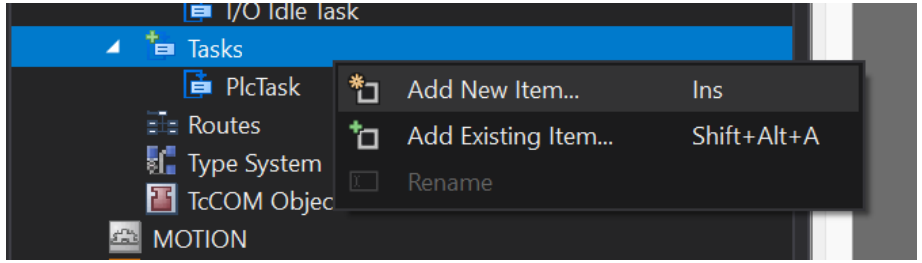


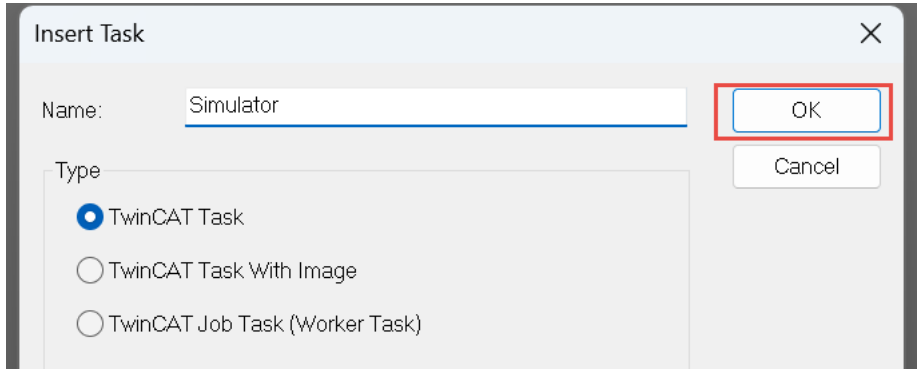




Create new task in TwinCAT

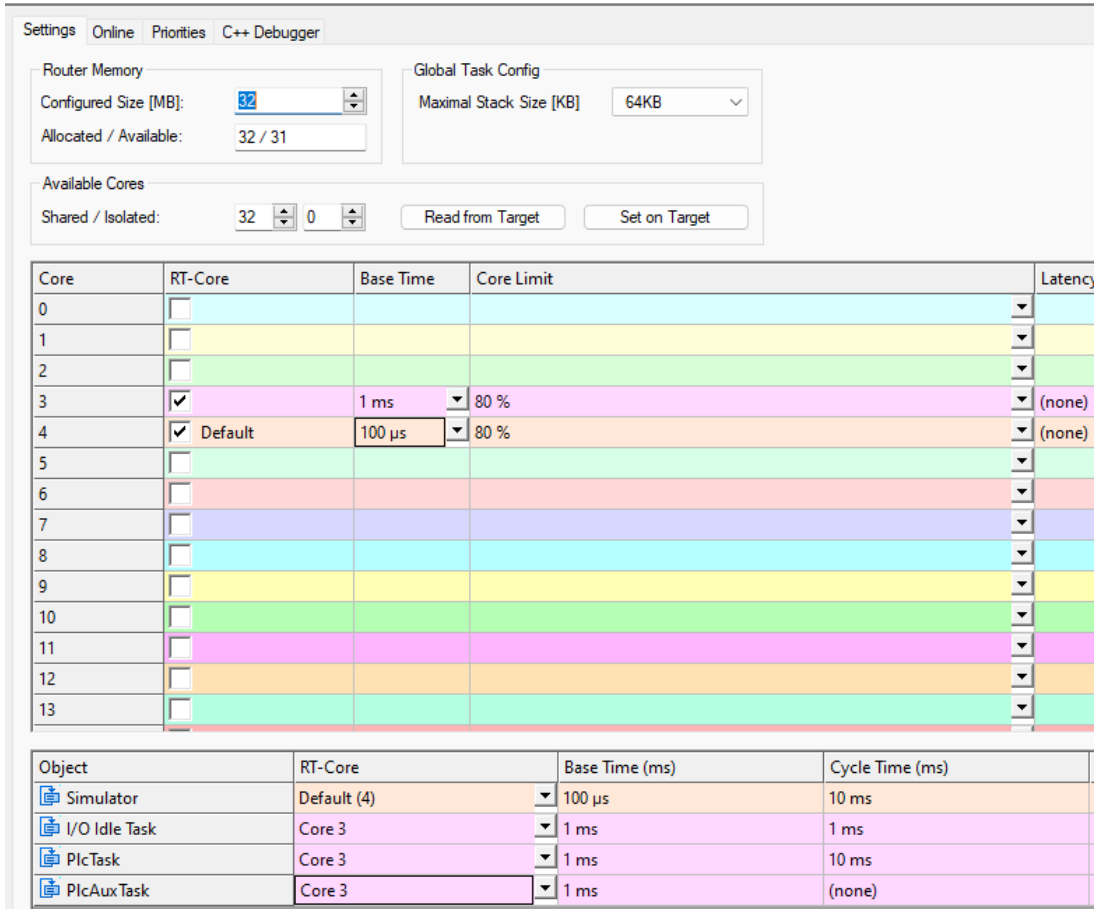
1. **Create new task** and name it





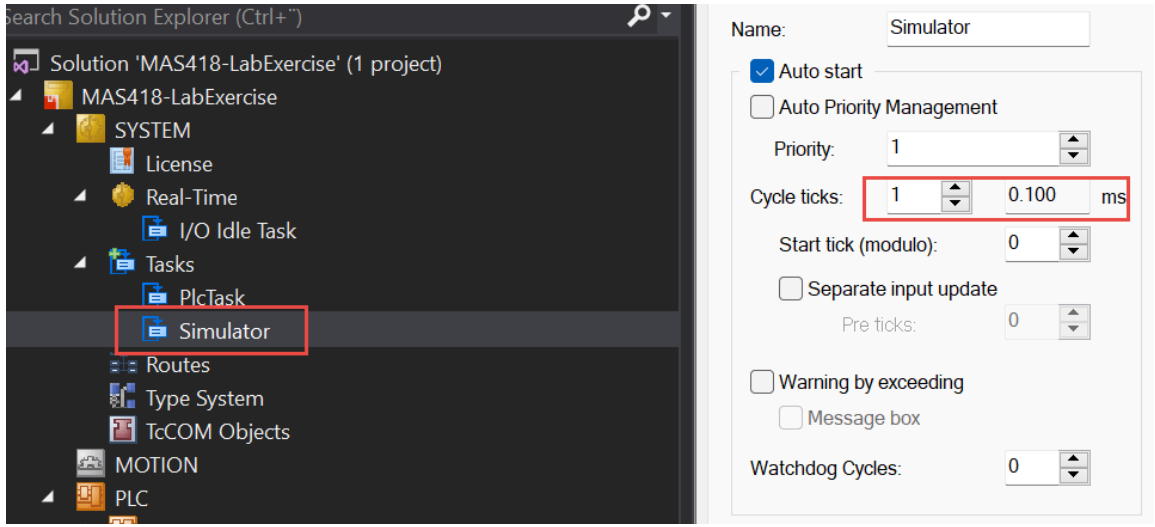
Create new task in TwinCAT

1. **Create new task** and name it
2. In real time window – **assign a new core** with base time 100 microseconds



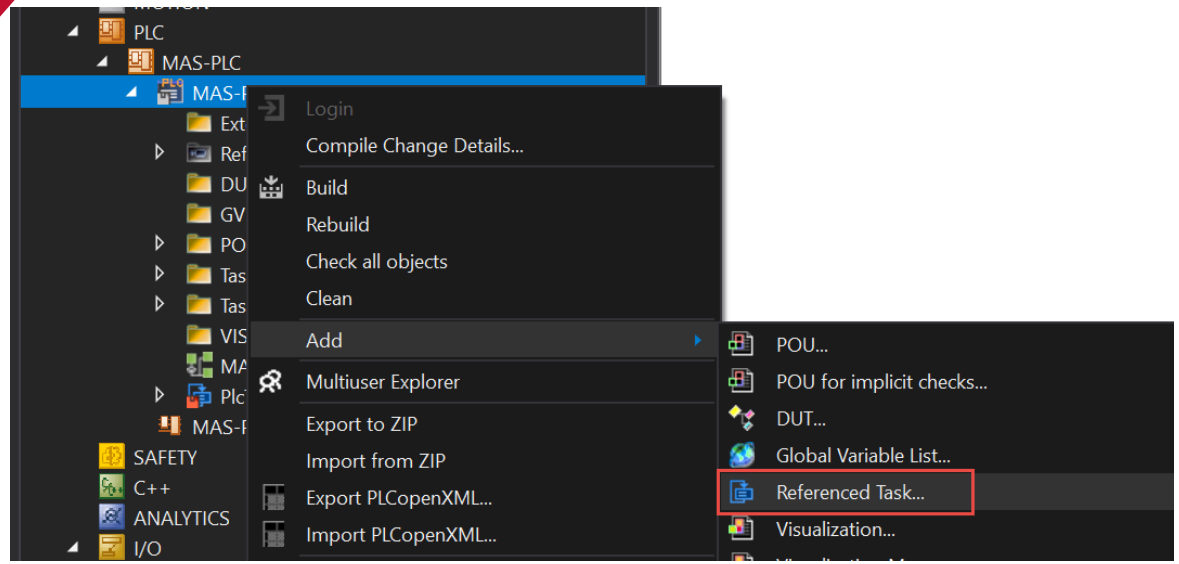
Create new task in TwinCAT

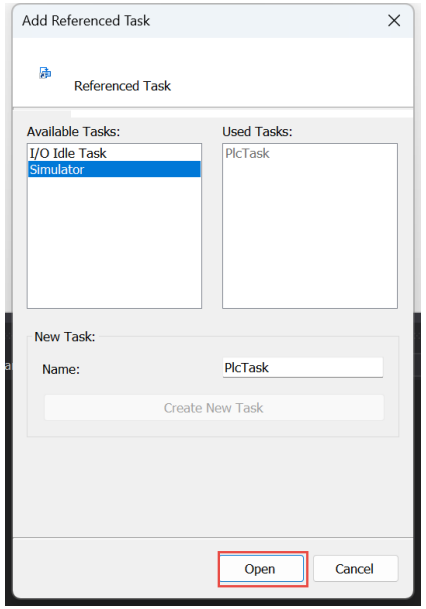
1. **Create new task** and name it
2. In real time window – **assign a new core** with base time 100 micro seconds
   - must be isolated if using the virtual machine
3. Adjust cycle time (ticks) to 0.1ms



Create new task in TwinCAT

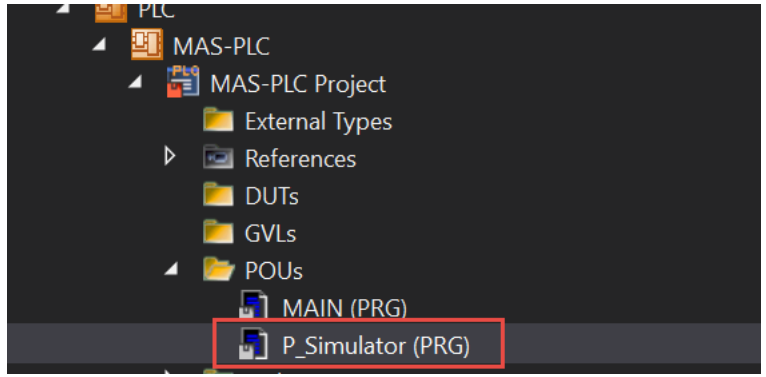
1. **Create new task** and name it
2. In real time window – **assign a new core** with base time 100 micro seconds
   - must be isolated if using the virtual machine
3. Adjust cycle time (ticks) to 0.1ms
4. Add mew Referenced Task and select the new task





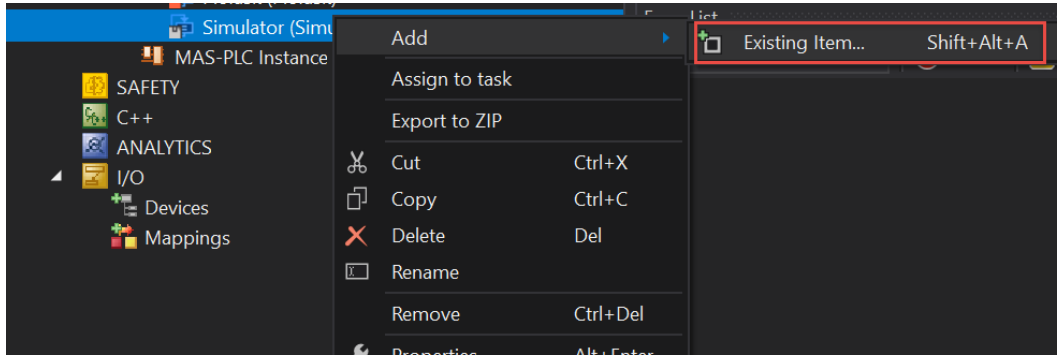
Create new task in TwinCAT

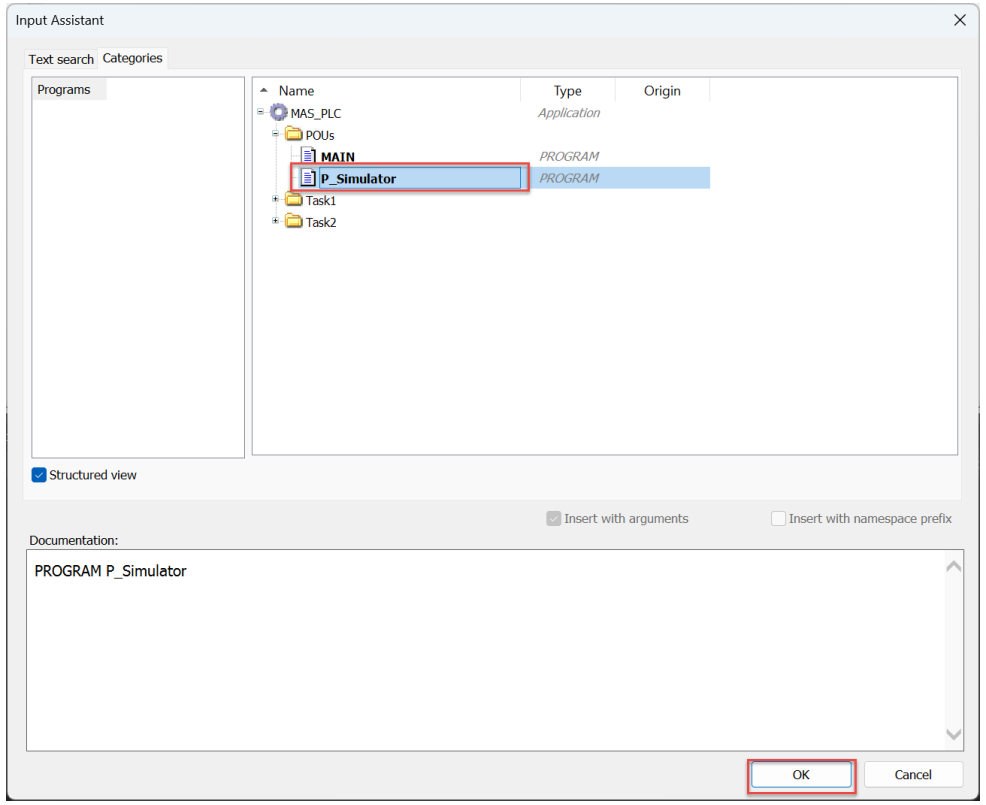
1. **Create new task** and name it
2. In real time window – **assign a new core** with base time 100 micro seconds
   - must be isolated if using the virtual machine
3. Adjust cycle time (ticks) to 0.1ms
4. Add mew Referenced Task and select the new task
5. Create a new Program



Create new task in TwinCAT

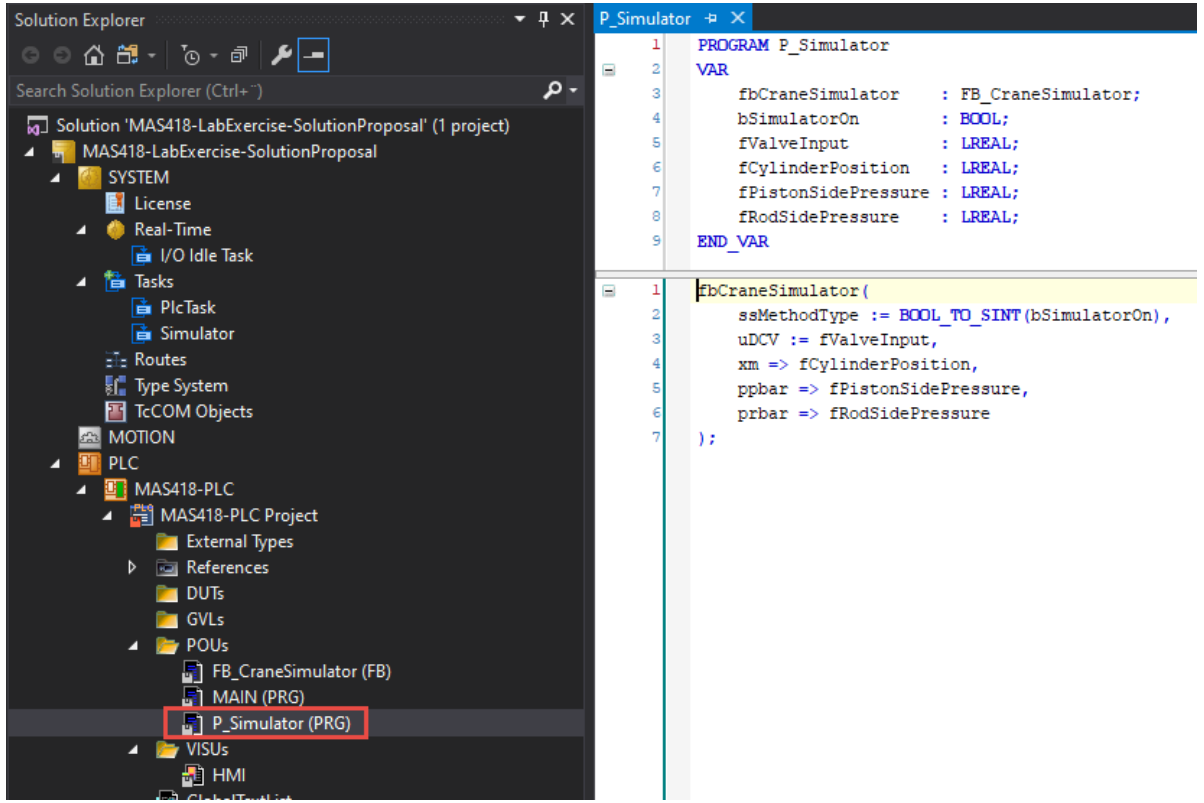
1. **Create new task** and name it
2. In real time window – **assign a new core** with base time 100 micro seconds
   - must be isolated if using the virtual machine
3. Adjust cycle time (ticks) to 0.1ms
4. Add mew Referenced Task and select the new task
5. Create a new Program
6. Assign the new Program to the task



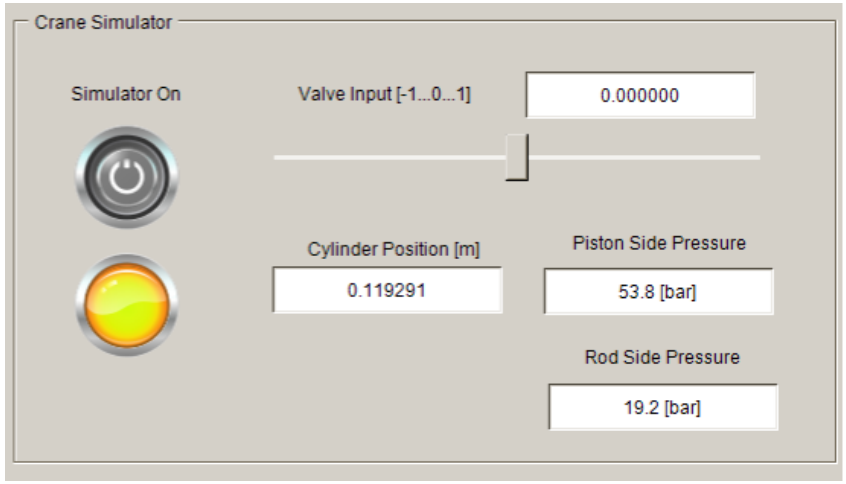


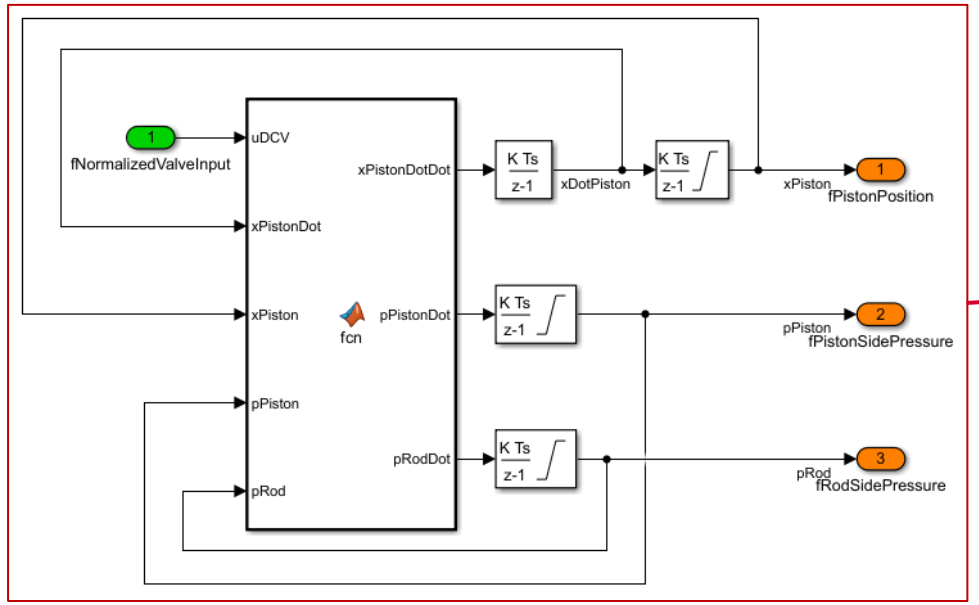
Create new task in TwinCAT

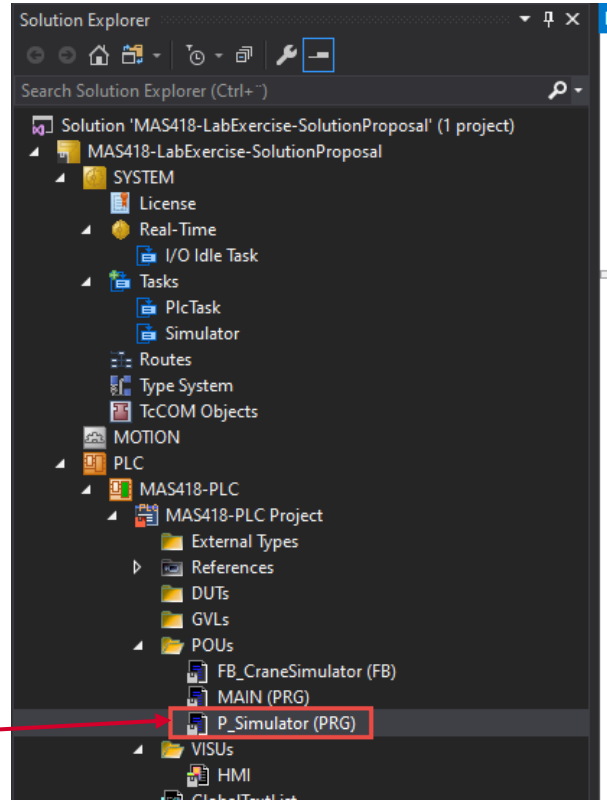
1. **Create new task** and name it
2. In real time window – **assign a new core** with base time 100 micro seconds
   - must be isolated if using the virtual machine
3. Adjust cycle time (ticks) to 0.1ms
4. Add mew Referenced Task and select the new task
5. Create a new Program
6. Assign the new Program to the task
7. Create a FB(s) for the simulator (or implement from Simulink PLC coder), instantiate it in the new program

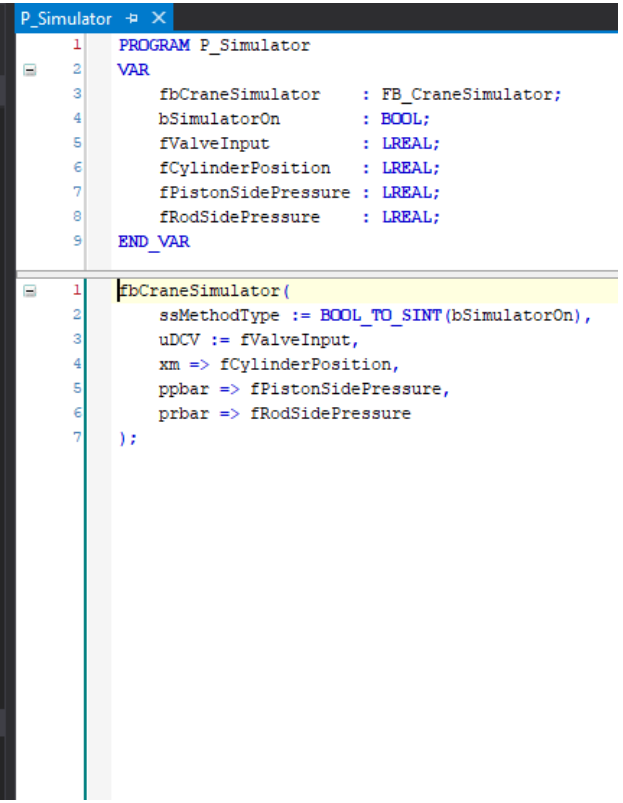


Create new task in TwinCAT









Part II: Structures & Functions

1. Structures
2. Functions
3. Parameter handling

UiA

Structures

```
aEventType : ARRAY[1..100] OF E_EventType;
aEventSeverity : ARRAY[1..100] OF TcEventSeverity;
aEventIdentity : ARRAY[1..100] OF UDINT;
aEventText : ARRAY[1..100] OF STRING(255);
aTimestamp : ARRAY[1..100] OF DATE_AND_TIME;
```

UiA

https://www.youtube.com/watch?v=zB_2x0NPMGc

Structures

- **Structure** is a used defined data type available in IEC 61131-3 that allows us to combine data items of **different** types

```
TYPE ST_Event :
STRUCT
    eEventType : E_EventType;
    eEventSeverity : TcEventSeverity;
    nEventIdentity : UDINT;
    sEventText : STRING(255);
    dtTimestamp : DATE_AND_TIME;
END_STRUCT
END_TYPE
```

```
aEventType : ARRAY[1..100] OF E_EventType;
aEventSeverity : ARRAY[1..100] OF TcEventSeverity
aEventIdentity : ARRAY[1..100] OF UDINT;
aEventText : ARRAY[1..100] OF STRING(255);
aTimestamp : ARRAY[1..100] OF DATE_AND_TIME;
```

```
VAR
    aEvents : ARRAY[1..100] OF ST_Event;
END_VAR
```

UiA

https://www.youtube.com/watch?v=zB_2x0NPMGc

Structures

- **Structure** is a used defined data type available in IEC 61131-3 that allows us to combine data items of **different** types

```
TYPE ST_Event :
STRUCT
    eEventType : E_EventType;
    eEventSeverity : TcEventSeverity;
    nEventIdentity : UDINT;
    sEventText : STRING(255);
    dtTimestamp : DATE_AND_TIME;
END_STRUCT
END_TYPE
```

```
VAR
    aEvents : ARRAY[1..100] OF ST_Event;
    sTempString : STRING(255);
END_VAR
```

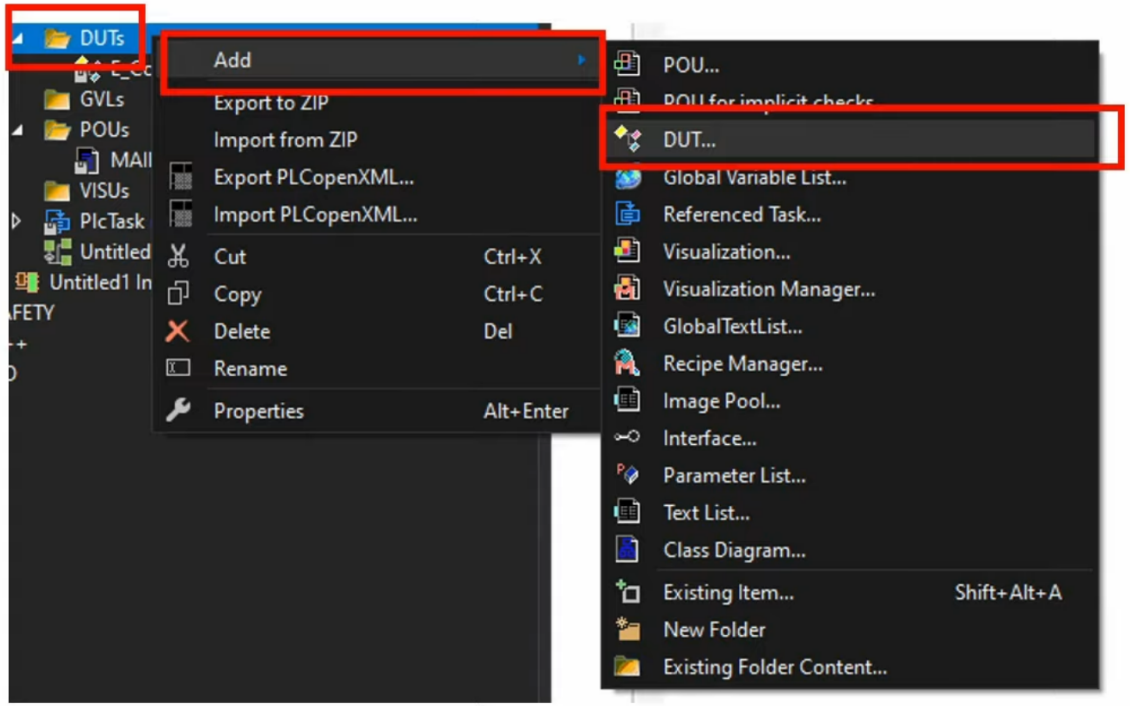
```
sTempString := aEvents[33].sEventText;
```

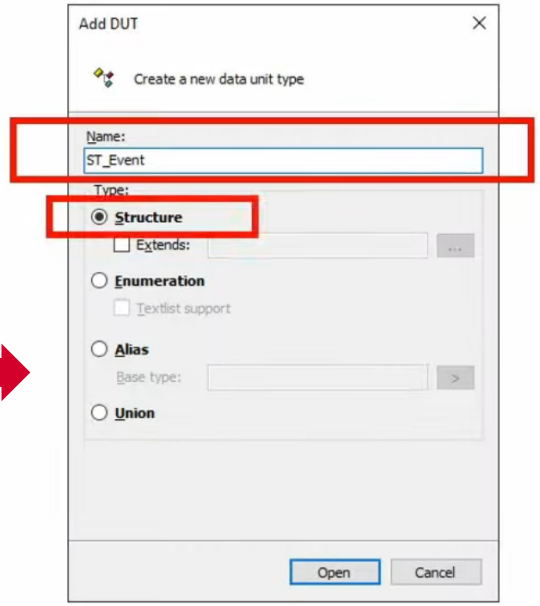
UiA

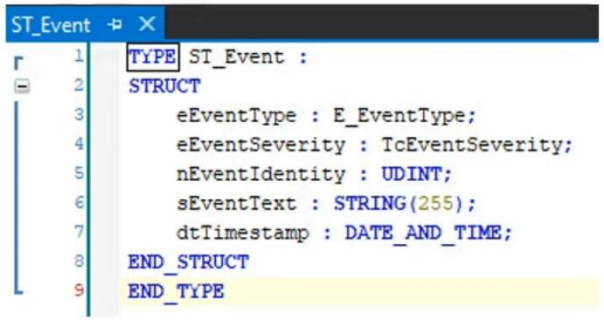
https://www.youtube.com/watch?v=zB_2x0NPMGc

Structures

- **How to create a struct:**





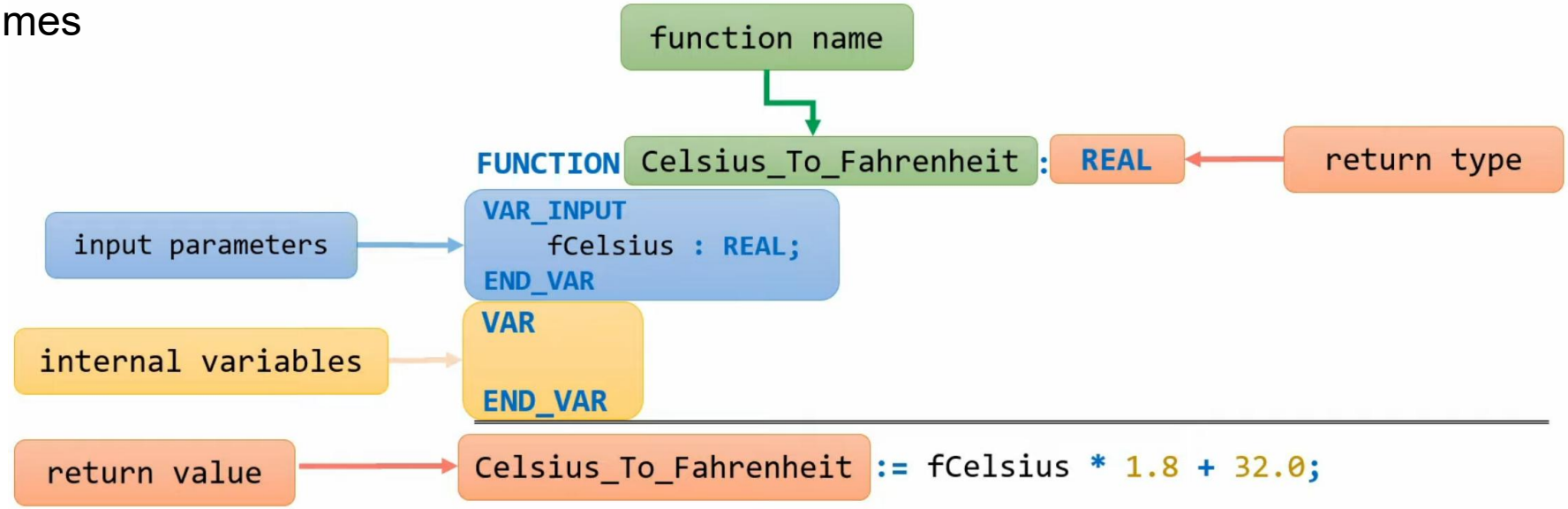


- Just as with enumerations the scope of **structures** is global

https://www.youtube.com/watch?v=zB_2x0NPMGc

Functions

- Functions are used to perform certain actions and they are important for reusing code
  - Define the code once, and you can use it many times




```
PROGRAM MAIN
VAR
    fFahrenheit : REAL;
END_VAR
```

41 ← `fFahrenheit := Celsius_To_Fahrenheit(fCelsius := 5.0);`

https://www.youtube.com/watch?v=zB_2x0NPMGc

Functions

```
FUNCTION Greater_Smaller
VAR_INPUT
    nNumber1 : INT;
    nNumber2 : INT;
END_VAR
VAR_OUTPUT
    nGreater : INT;
    nSmaller : INT;
END_VAR
```

output parameters

```
IF nNumber1 > nNumber2 THEN
    nGreater := nNumber1;
    nSmaller := nNumber2;
ELSE
    nGreater := nNumber2;
    nSmaller := nNumber1;
END_IF
```

```
PROGRAM MAIN
VAR
    nReturnGreater : INT;
    nReturnSmaller : INT;
END_VAR
```

```
Greater_Smaller(nNumber1 := 8,
                nNumber2 := 15,
                nGreater => nReturnGreater,
                nSmaller => nReturnSmaller);
```

nReturnGreater	nReturnSmaller
15	8

https://www.youtube.com/watch?v=zB_2x0NPMGc

Functions

```
FUNCTION SwapNums
VAR_IN_OUT
    nNumber1 : INT; 5
    nNumber2 : INT; 15
END_VAR
VAR
    nNumberTemp : INT;
END_VAR
```

input & output parameters at same time!

```
5  nNumberTemp := nNumber1; 5
15 nNumber1 := nNumber2; 15
5  nNumber2 := nNumberTemp; 5
```

```
PROGRAM MAIN
VAR
    nA : INT := 5;
    nB : INT := 15;
END_VAR
```

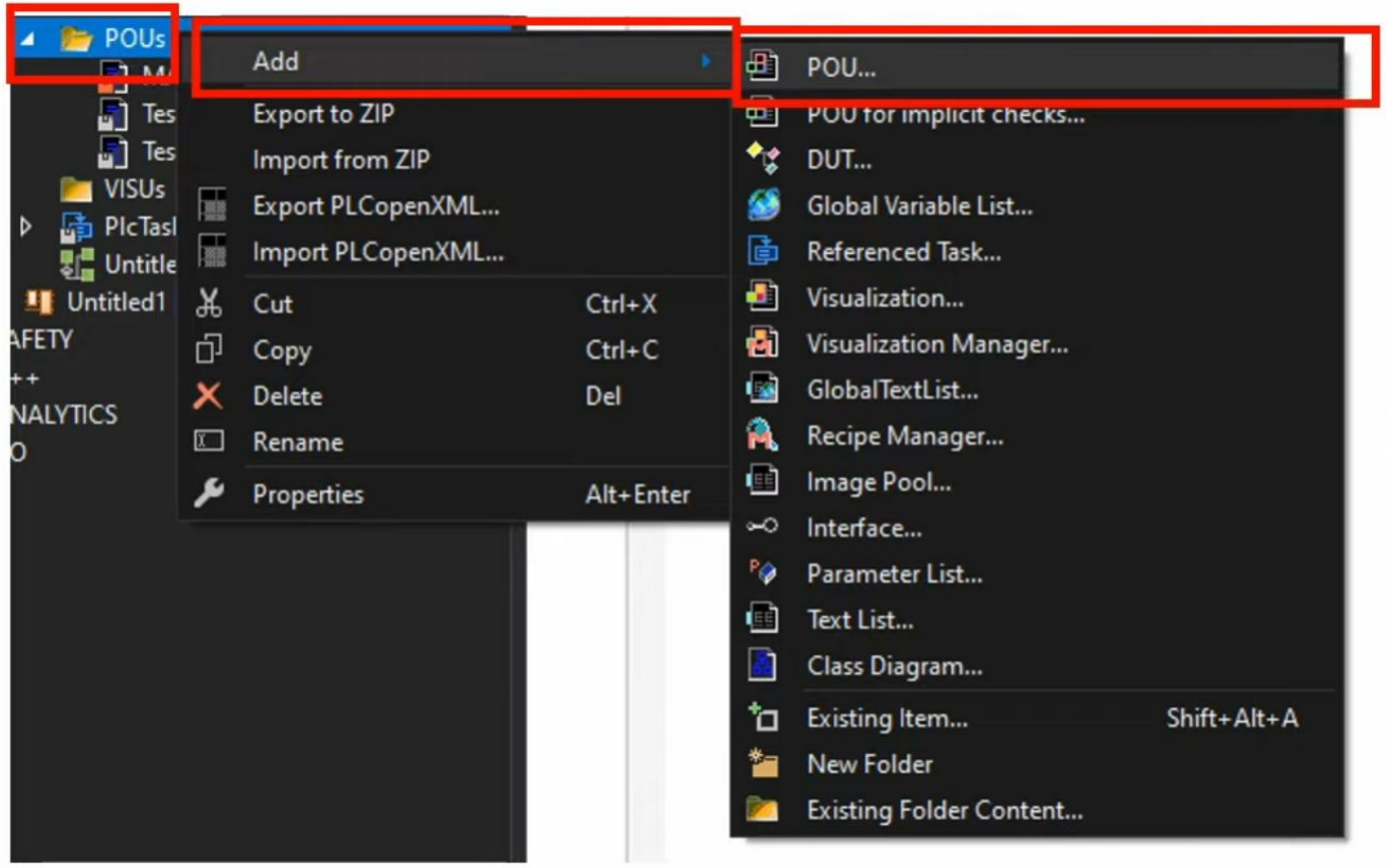
```
SwapNums(nNumber1 := nA,
         nNumber2 := nB);
```

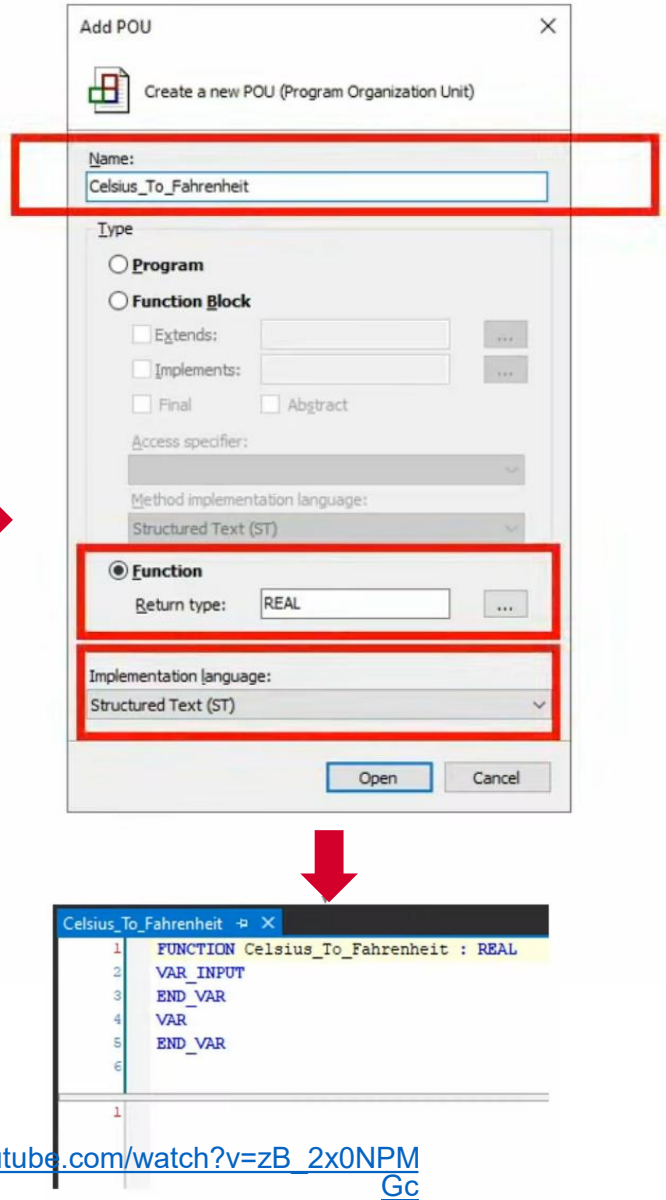
nA	nB
15	5

https://www.youtube.com/watch?v=zB_2x0NPMGc

Functions

- **To create a function:**

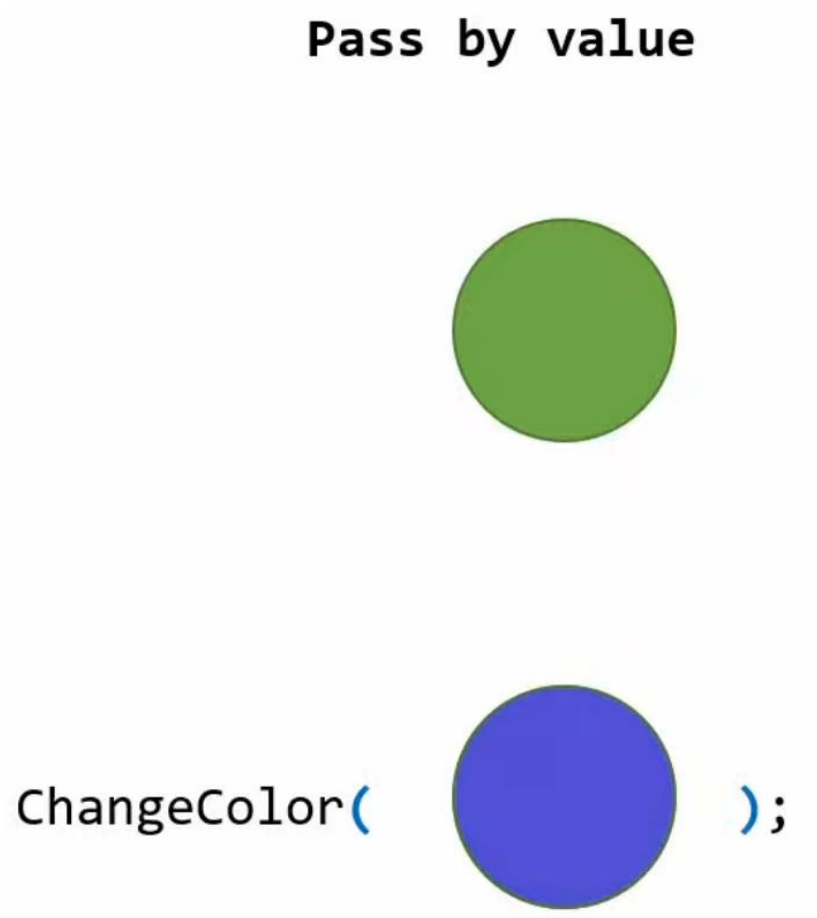


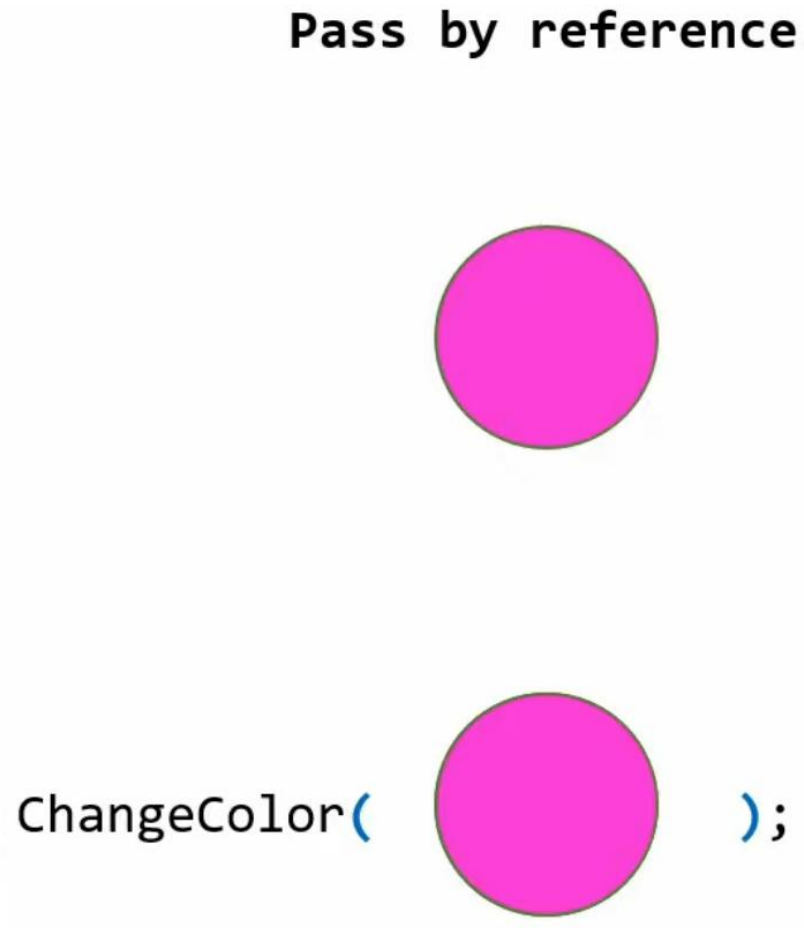


https://www.youtube.com/watch?v=zB_2x0NPMGc

Parameter handling

- When passing values into your function, you can do so either by value or by reference





https://www.youtube.com/watch?v=zB_2x0NPMGc

Parameter handling

Pass by value

IEC 61131-3

```
FUNCTION UpdateEventTimestampWithSystemTime
VAR_INPUT
    stEvent : ST_Event;
END_VAR
VAR_OUTPUT
    stEventOut : ST_Event;
END_VAR
```

```
stEventOut := stEvent;
stEventOut.dtTimestamp := …(GetSystemTime…)
```

Pass by reference

IEC 61131-3

```
FUNCTION UpdateEventTimestampWithSystemTime
VAR_IN_OUT
    stEvent : ST_Event;
END_VAR
```

```
FUNCTION UpdateEventTimestampWithSystemTime
VAR_INPUT
    stEvent : REFERENCE TO ST_Event;
END_VAR
```

```
stEvent.dtTimestamp := …(GetSystemTime…)
```

https://www.youtube.com/watch?v=zB_2x0NPMGc

Parameter handling

- **When should we use pass by value and when should we use pass by reference?**
  - Pass by value means you are making a copy in memory of the parameters value that is passed in a copy of the contents of the actual parameter
  - The penalty for this can get quite extensive if the data that has to be copied is big
- In general:
  1. Pass by **value** when the function does not want to modify the parameter and the value is easy to copy (small parameter → ints, floats, bools, etc. → most a few words)
  2. Pass by **value** when the function does not want to modify the parameter and the value is expensive to copy (structure events example etc.)
  3. Pass by **reference** when the function does want to modify the parameter and the value is expensive to copy

https://www.youtube.com/watch?v=zB_2x0NPMGc

Part III: Control Instructions

1. Introduction
2. Conditional statements
3. CASE instruction
4. FOR loops
5. WHILE loops

Introduction

- **Instructions** are used to control the flow of our program
- **In this part we will go through the following:**
  - **Conditional (IF/ELSE) statements**
  - **CASE instruction**
  - **FOR loops**
  - **WHILE loops**

Conditional statements

- Use **IF** to specify a block of code to be executed, if a specified conditions is true

```
IF condition THEN
    // block of code to be executed if the condition is true
END_IF
```

- Use **ELSE** to specify a block of code to be executed, if the same condition is false

```
IF condition THEN
    // block of code to be executed if the condition is true
ELSE
    // block of code to be executed if the condition is false
END_IF
```

- Use **ELSIF** to specify a new condition to test, if the first condition is false

```
IF condition1 THEN
    // block of code to be executed if condition1 is true
ELSIF condition2 THEN
    // block of code to be executed if condition1 is false and condition2 is true
ELSE
    // block of code to be executed if condition1 is false and condition2 is false
END_IF
```

- Use **CASE** to specify many alternative blocks of code to be executed

UiA

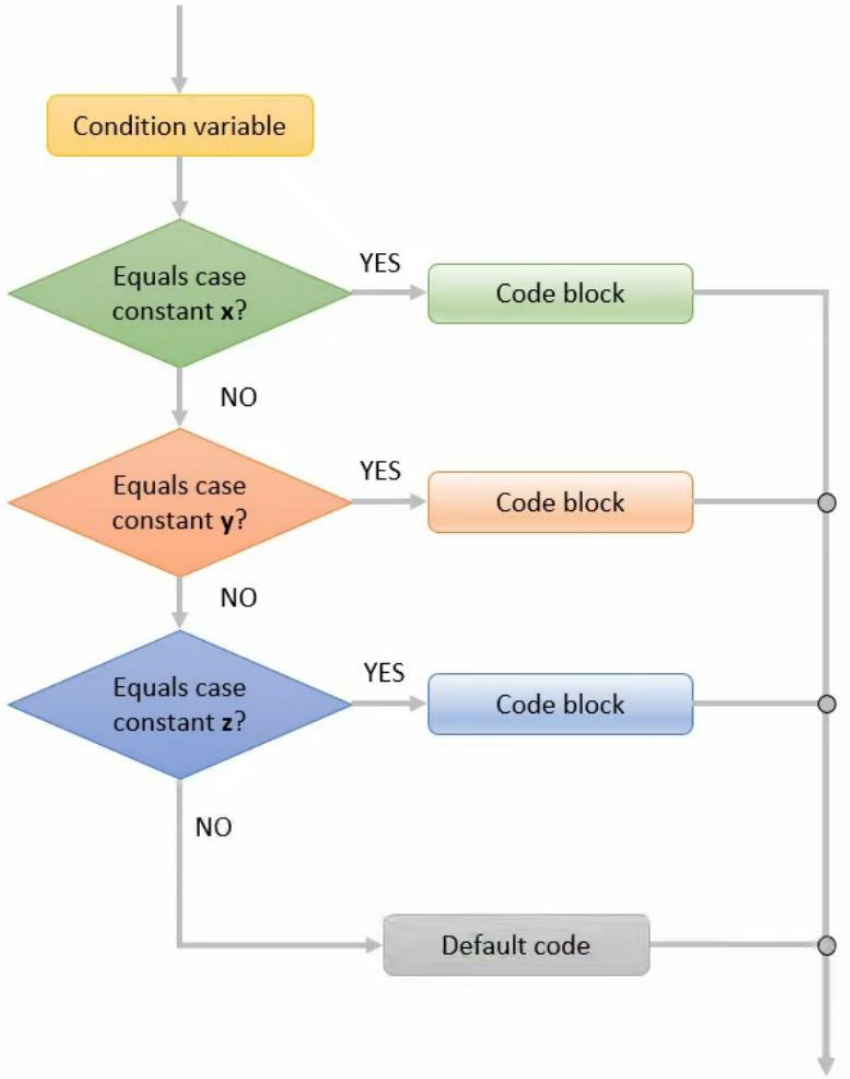
https://www.youtube.com/watch?v=pnLgRNb09qE&list=PLimaF0nZKYHz3I3kFP4myaAYjmYk1SowO&index=8&t=1s

CASE instruction

- The **CASE** statement is used to perform different action based on a condition variable

```
CASE variable OF
    x :
        // block of code
    y :
        // block of code
    z :
        // block of code
ELSE
    // block of code
END_CASE
```

- The value of the variable is compared with the value of case
- If there is a match, the associated block of code is exec
- If there is no match the **ELSE** code block is executed.



https://www.youtube.com/watch?v=pnLgRNb09qE&list=PLimaF0nZKYHz3I3kFP4myaAYjmYk1SowO&index=8&t=1s

FOR loops

- **FOR**-loops can execute a block of code a number of times

Instead of writing:

```
VAR
   nSum : INT;
   aArray : ARRAY[1..5] OF INT;
END_VAR
```

```
nSum := nSum + aArray[1];
nSum := nSum + aArray[2];
nSum := nSum + aArray[3];
nSum := nSum + aArray[4];
nSum := nSum + aArray[5];
```

You can write:

```
VAR
   nSum : INT;
   nCounter : INT;
   aArray : ARRAY[1..5] OF INT;
END_VAR
```

Optional

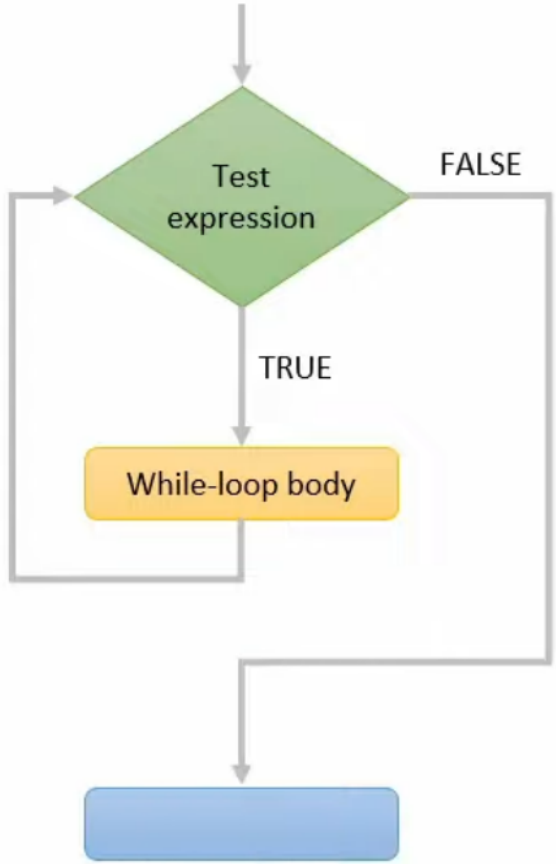
```
FOR nCounter := 1 TO 5 BY 1 DO
    nSum := nSum + aArray[nCounter];
END_FOR
```

https://www.youtube.com/watch?v=pnLgRNb09qE&list=PLimaF0nZKYHz3I3kFP4myaAYjmYk1SowO&index=8&t=1s

WHILE loops

- The **WHILE**-loop loops through a block of code as long as a specified condition is true

```
VAR
   nSum : INT;
   nCounter : INT := 1;
   aArray : ARRAY[1..5] OF INT;
END_VAR

WHILE nCounter < 6 DO
    nSum := nSum + aArray[nCounter];
    nCounter := nCounter + 1;
END_WHILE

// block of code
```

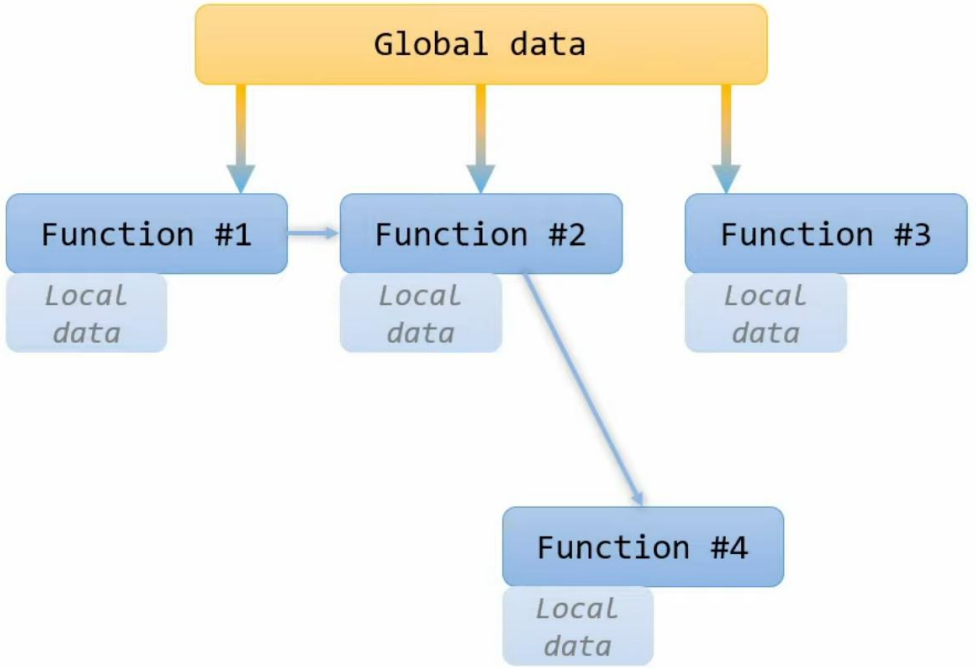


https://www.youtube.com/watch?v=pnLgRNb09qE&list=PLimaF0nZKYHz3I3kFP4myaAYjmYk1SowO&index=8&t=1s

Summary

Summary

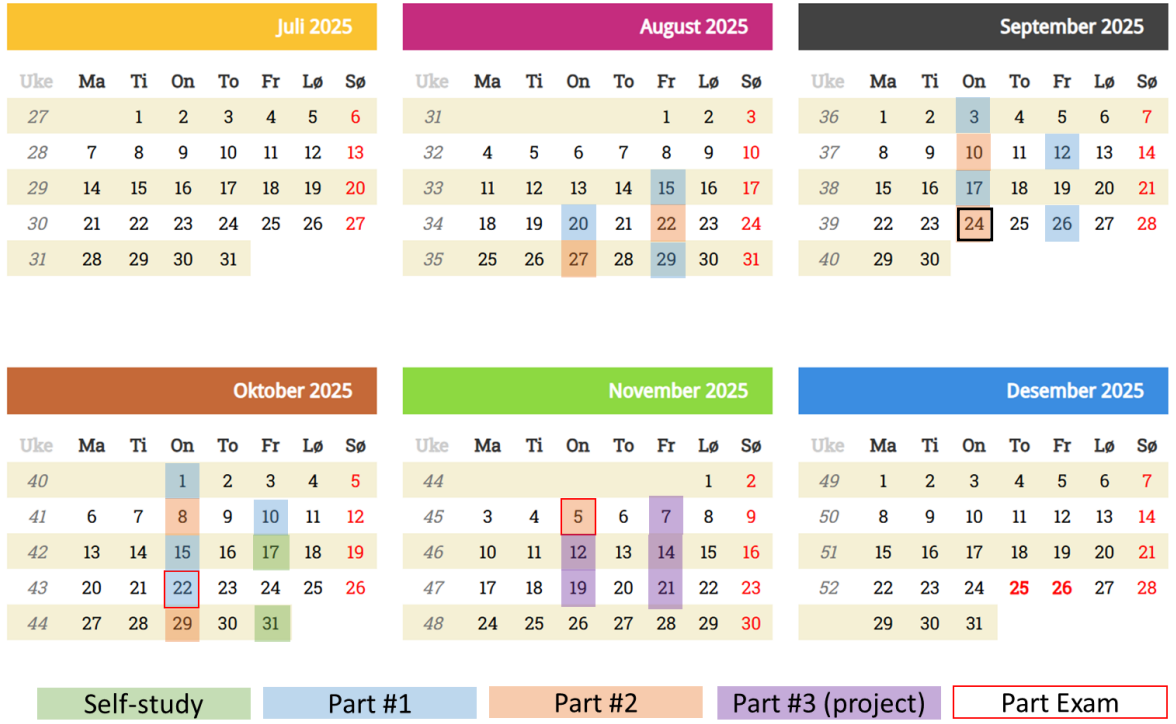
- In **Part I** we looked into the Green Crane application and the simplified hydraulic cylinder simulator in MATLAB-Simulink and how to export it using the PLC coder and run it in a separated task.
- In **Part II** we looked into how we can organize data into structures and actually make something useful with that data using functions
- In **Part III** we looked into **Interfaces**
  - Conditional statements
  - CASE instruction
  - FOR loops
  - WHILE loops



Next Lecture

Object oriented PLC Programming:

I. Function Blocks
II. Interfaces



Juli 2025

Uke	Ma	Ti	On	To	Fr	Lø	Sø
27		1	2	3	4	5	6
28	7	8	9	10	11	12	13
29	14	15	16	17	18	19	20
30	21	22	23	24	25	26	27
31	28	29	30	31			

August 2025

Uke	Ma	Ti	On	To	Fr	Lø	Sø
31					1	2	3
32	4	5	6	7	8	9	10
33	11	12	13	14	15	16	17
34	18	19	20	21	22	23	24
35	25	26	27	28	29	30	31

September 2025

Uke	Ma	Ti	On	To	Fr	Lø	Sø
36	1	2	3	4	5	6	7
37	8	9	10	11	12	13	14
38	15	16	17	18	19	20	21
39	22	23	24	25	26	27	28
40	29	30					

Oktober 2025

Uke	Ma	Ti	On	To	Fr	Lø	Sø
40			1	2	3	4	5
41	6	7	8	9	10	11	12
42	13	14	15	16	17	18	19
43	20	21	22	23	24	25	26
44	27	28	29	30	31		

November 2025

Uke	Ma	Ti	On	To	Fr	Lø	Sø
44						1	2
45	3	4	5	6	7	8	9
46	10	11	12	13	14	15	16
47	17	18	19	20	21	22	23
48	24	25	26	27	28	29	30

Desember 2025

Uke	Ma	Ti	On	To	Fr	Lø	Sø
49	1	2	3	4	5	6	7
50	8	9	10	11	12	13	14
51	15	16	17	18	19	20	21
52	22	23	24	25	26	27	28
	29	30	31				

Self-study | Part #1 | Part #2 | Part #3 (project) | Part Exam

Lab exercise

Lab Exercise #2.2 – Procedural-oriented PLC programming